OSGuard

Risk-Aware Operating-System Update Orchestration

An explainable OS-update risk assessment and rollout simulation platform – educational project

Field	Entry
Team name	NOVA
Team members	Anirudh Koppuravuri – 24BCE0490 Kalivarapu Prabhas – 24BCE2279 Barath Sornaraj – 24BCE2599
Institution	Vellore Institute of Technology
Course / course code	[EDITABLE: course name and code]
Lecturer / evaluator	[EDITABLE: lecturer name]
Slot / semester	[EDITABLE]
Submission date	[EDITABLE: dd Month yyyy]
Video link	[EDITABLE: add after upload – see Section 14]

Academic prototype. All machines, patches, vulnerabilities and outcomes are SYNTHETIC; results are simulation outputs, not real-world measurements. OSGuard never modifies the host operating system, installs patches or runs privileged operations. OSGuard is not xv6, not a kernel and not a real multi-OS patching agent. Developed for educational purposes (coursework) only.

Table of contents

1. Submission requirements and traceability 4
2. Abstract, objectives and scope 5
2.1 Abstract 5
2.2 Problem statement 5
2.3 Objectives 5
2.4 Scope and contributions 5
3. Operating-system context and concepts 5
3.1 Host and runtime environment (read from the machine) 5
3.2 The simulated fleet 6
3.3 Operating-systems concepts demonstrated 6
3.4 What OSGuard is not 6
4. Industry problem analysis and existing solutions 6
4.1 OSGuard's defensible contribution 7
5. System design 7
5.1 Architecture and data flow 7
5.2 Components 7
5.3 Database structure 8
5.4 API design 8
5.5 Technology stack (installed versions) 8
6. Algorithms and safety gates 9
6.1 Urgency score 9
6.2 Compatibility risk and uncertainty 9
6.3 Readiness rules and safety gates 9
6.4 Rollout state transitions 10
6.5 Worked examples (from the running application, seed 2026) 10
6.6 Failure model (hidden ground truth of the simulation) 10
6.7 Metrics 10
7. Implementation verification 11
7.1 Commands executed and results 11
7.2 Evidence that unsafe states cannot silently pass 11
7.3 Defect found and fixed 12
7.4 Vitest and build logs 12
8. Experiments 12
8.1 Methodology and reproducible configuration 12
8.2 Results (simulation output) 13

8.3 Limitations....13
9. Native debugging with GDB....13
9.1 Applicability....13
9.2 Build and cross-check....13
9.3 GDB session (real output, GDB 17.1, WSL2 Ubuntu 26.04)....14
9.4 What the session shows....15
10. Screenshots (captured from the running application)....15
11. Viva questions and answers....18
12. Conclusion and future work....19
13. References....19
14. Submission checklist and video package....20
Appendix A. Source listings....21
Appendix B. Start-up commands (Windows)....38

1. Submission requirements and traceability

Assignment title as given: “Mini Project: Modifying a Small OS Kernel”. OSGuard is an OS-update orchestration simulator for educational purposes; it does not modify an operating-system kernel (see Section 3.4). The detailed rubric (“See Rubrics”) was not available when this report was prepared, so the rows below follow the written submission instructions. Status values are conservative: an item is only “Complete” if it was verified by a command or file listed here.

Table 1 – Requirements traceability

#	Requirement	Implementation / location	Evidence	Status
1	Single PDF with all work	This document (DOCX + PDF built from the same source)	docs/OSGuard_Project_Report.pdf (main) and docs/OSGuard_Project_Report_full_listings.pdf	Complete
2	Answers to questions	Section 11 (viva Q&A). The assignment’s own question list was not supplied	Section 11	Partly – team to add the lecturer’s exact questions
3	Code listings	Appendix A (full essential logic with hashes); the shorter main edition has key excerpts only	Files hashed at build time	Complete
4	GDB session logs	Native diagnostic harness native/rollout_gate.c debugged in GDB 17.1 (Section 9)	native/gdb_session.log, Section 9	Complete for the C harness only; Python/TypeScript are not GDB-debugged
5	Screenshots	Section 10: dashboard, fleet, risk assessment, rollout success, fault containment, comparison, analytics, audit, sources	docs/screenshots/*.png (captured from the running app)	Complete
6	Modify a small OS kernel (assignment title)	Not implemented: OSGuard is a simulator running on Windows 11, not a kernel and not xv6	Section 3	NOT satisfied by this project
7	Screen-recorded walkthrough; every member explains a portion, runs the program, own voice	docs/VIDEO_WALKTHROUGH_SCRIPT.md and docs/scripts/ (one script per member, sequential)	None yet – must be recorded by the team	Unresolved – team action
8	Video on Clipchamp/Google Drive (shared with the evaluator) or YouTube Unlisted	Checklist in Section 14	—	Unresolved – team action
9	List mapping each member to a timestamp range	Template in Section 14 with member names	—	Template only – fill after recording
10	Readiness for follow-up questions	Section 11 and the scripts’ Q&A	—	Prepared; team must study it
11	Working implementation: risk engine, blockers, fleet, rollout simulator, thresholds, fault injection, audit, experiments, APIs, UI	Sections 5–8	56 backend tests, 12 frontend tests, type-check and production build all passed (Section 7)	Complete

2. Abstract, objectives and scope

2.1 Abstract

Operating-system updates close security holes but also break drivers, applications and services. OSGuard is a prototype that decides, for each machine in a synthetic fleet, whether an update is urgent, whether it is safe enough to deploy, and how to roll it out in stages so that a bad update is contained. It separates three questions that are often blended into one score: how urgent is the patch (severity, exposure, criticality, age), what evidence says it will fail (failed compatibility checks), and what evidence is missing (unknown checks, absent telemetry). Missing or failed mandatory evidence always blocks deployment. Eligible machines are updated through a canary and staged batches with health thresholds and automatic halting, every decision is written to an audit log in SQLite, and reproducible experiments compare the staged strategy with an all-at-once baseline.

2.2 Problem statement

How can an organisation prioritise security patches while reducing compatibility risk and service disruption, using evidence it can inspect and explain, and how can that policy be evaluated without risking real systems?

2.3 Objectives

- Compute an explainable urgency score from security severity, exposure, operational criticality and patch age with configurable weights.
- Track compatibility risk (failed evidence) and uncertainty (missing evidence) separately, and never treat missing or failed mandatory evidence as safe.
- Simulate canary and staged rollouts with latency/error thresholds, fault injection and automatic halting, recording each decision and its reason.
- Compare staged and all-at-once strategies reproducibly on blast radius, completion time, simulated exposure, unnecessary delays and safety violations.
- Persist decisions, rollout events, triggered rules and experiment results, and allow filtering and export.

2.4 Scope and contributions

In scope: a deterministic synthetic fleet of 120 machines, a rule-based risk engine, a simulator, six scenarios, a React/TypeScript interface with an interactive 3D fleet map, and a native C diagnostic harness used for GDB debugging. Out of scope: real patch installation, real vendor telemetry, machine learning, authentication, multi-user operation.

- A transparent readiness gate (rules B1–B6) that keeps negative evidence and missing evidence distinct.
- A resumable rollout state machine with explicit allowed transitions and an auditable halt decision.
- A paired-seed comparison method in which both strategies face identical hidden outcomes.
- A cross-check of the safety rules in an independent native implementation (0 mismatches over 120 machines, Section 9).

3. Operating-system context and concepts

3.1 Host and runtime environment (read from the machine)

Table 2 – Execution environment

Item	Value
Host OS	Microsoft Windows 11 Home Single Language, version 10.0.26300 (build 26300), 64-bit (AMD64)
CPU / memory	13th Gen Intel Core i5-13450HX / 15.7 GB

Item	Value
Python / Node.js / npm	3.14.7 / v22.13.1 / 10.9.2
SQLite library (bundled with Python)	3.50.4
Shells	PowerShell 5.1, Git Bash
WSL distribution used only for the C/GDB demonstration	WSL2 Ubuntu 26.04 LTS, kernel 6.18.33.2-microsoft-standard-WSL2
GDB / GCC (inside WSL)	GNU gdb (Ubuntu 17.1-2ubuntu1) 17.1 / gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0

OSGuard itself (Python/FastAPI backend and React/TypeScript frontend) ran natively on Windows 11. Only the diagnostic harness in Section 9 was compiled and debugged in the WSL2 Ubuntu environment.

3.2 The simulated fleet

The fleet contains 120 synthetic machine records. Each carries an OS family and version as a text label: Windows Server (2019, 2022, 2025), Windows 11 (23H2, 24H2), Ubuntu (22.04, 24.04), RHEL (8.10, 9.4), Amazon Linux (2023), macOS (14, 15) and ICS/Embedded (RTOS 4.2, HMI 7.1). These are not independently running operating systems: no guest OS is installed, booted, emulated or patched. Patch identifiers (for example SYN-WS-2026-10), CVSS-style scores and the "vendor hold" are invented.

3.3 Operating-systems concepts demonstrated

- **Patch and update management:** urgency from severity, exposure, criticality and age; restart-required versus live-patchable updates (the live-patch idea follows AWS's documented Amazon Linux 2023 kernel live patching, Section 4).
- **Compatibility and dependency risk:** driver, application and backup evidence gate each update; a safeguard-hold style block models vendor-known incompatibility.
- **Reliability and recovery:** health thresholds, automatic halt, rollback of unhealthy machines, and a rollout state machine (planned → running → completed | halted | aborted).
- **Concurrency:** the web server runs handlers on worker threads; an intermittent defect where an SQLite connection crossed threads was found and fixed (Section 7.3).
- **Native debugging:** stack frames, conditional breakpoints, watchpoints and return-value inspection on a C program with GDB (Section 9).

3.4 What OSGuard is not

- Not xv6, not a teaching kernel, not QEMU-based.
- Not a real patching agent and not a replacement for commercial tools.
- Not using machine learning or AI; all logic is deterministic rules and arithmetic.
- Not tied to real companies' data: all numbers are synthetic or simulated.

4. Industry problem analysis and existing solutions

On 26 October 2026 the text of each page below was fetched and the statements were checked against it. Linked sub-pages and PDFs were not read; the Apple and CISA entries rest on an index or table of contents only. Entries marked "Our observation" are the team's own comparison, not vendor claims.

Table 3 – Existing solutions checked against the cited pages

Platform	What the cited page says	Limitation / our observation

Platform	What the cited page says	Limitation / our observation
Microsoft Windows – safeguard holds	Microsoft uses quality and compatibility data to identify issues that could cause a feature update to fail or roll back, and may hold the update for affected devices until a fix is verified. Windows Update for Business reports list active holds; Autopatch-managed devices also benefit.	Holds only affect devices using the Windows Update service; admins using other channels (media, WSUS) must watch known issues themselves. Our observation: holds reflect vendor-known issues, not your own application inventory.
AWS Systems Manager Patch Manager	Patch baselines per supported OS type; patch policies, maintenance windows or on-demand “Patch now”; compliance reporting.	Our observation: the page describes policy and scheduling, not per-machine compatibility-evidence scoring.
AWS – Amazon Linux 2023 kernel live patching	Applies specific security and critical-bug kernel patches to a running kernel without rebooting; can improve availability until reboot.	Some issues cannot be fixed by live patching and are only shipped in full kernel updates; the page lists functional limitations while a live patch is applied.
Google Cloud – GKE upgrade best practices	Covers release channels, maintenance windows and exclusions, surge and blue-green node upgrade strategies, Pod Disruption Budgets, version-skew limits, and testing changes in environments separate from production.	Scope is GKE clusters, not arbitrary fleets. Our observation: workload-specific go/no-go decisions are left to the operator.
Apple Platform Deployment	The guide has a “Manage software updates” section: about software updates, the update process, installing and enforcing updates, and testing updates with the AppleSeed for IT beta programme.	Only the table of contents was read; detailed capabilities are on sub-pages not reviewed.
CISA – ICS Recommended Practices	The page indexes ICS cybersecurity documents, including “Recommended Practice for Patch Management of Control Systems” and “Improving ICS Cybersecurity with Defense-in-Depth Strategies”.	Only the index was read; the PDFs were not reviewed. These are guidance documents, not an orchestration tool.

4.1 OSGuard’s defensible contribution

OSGuard does not claim to outperform any of these products and does not replace them. Its contribution is smaller and specific: it is a transparent, inspectable model of an evidence-gated rollout in which urgency, failed evidence and missing evidence are kept separate, every recommendation lists the rules behind it, and the safety policy can be exercised against reproducible faults without touching real systems. It is a teaching and evaluation prototype.

5. System design

5.1 Architecture and data flow

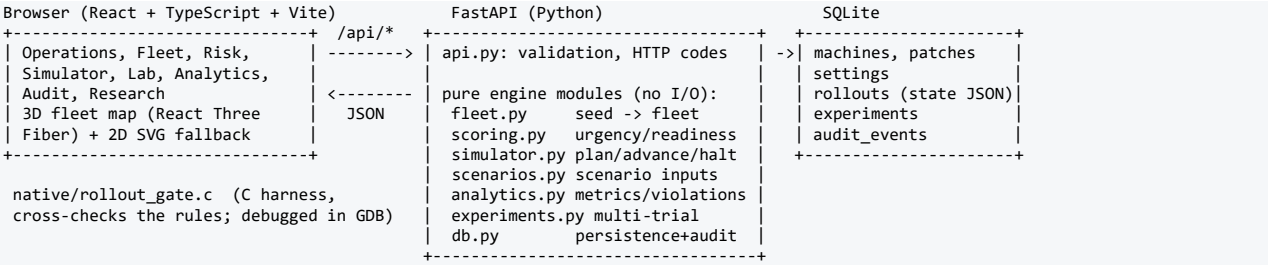


Figure 1 – Architecture of OSGuard (text diagram)

Data flow for a staged rollout: (1) the UI posts a rollout request; (2) the API loads machines and the scoring configuration from SQLite; (3) a scenario, if chosen, rewrites a deep copy of the fleet; (4) the engine assesses every machine and plans batches; (5) each “advance” runs one batch and returns events and metrics; (6) the full rollout state and any new audit events are saved; (7) the UI renders batches, metrics and decisions.

5.2 Components

Table 4 – Component responsibilities

Component	Responsibility

Component	Responsibility
fleet.py	Deterministic generator of 120 machines and the synthetic patch catalogue from a seed (default 2026)
scoring.py	Urgency score, compatibility risk, uncertainty, readiness rules B1–B6, recommended action with explanations
simulator.py	Failure model, planning (eligibility, ordering, canary choice), batch execution, health evaluation, halt, abort, state machine
scenarios.py / experiments.py	Six reproducible scenarios; multi-trial paired comparison of both strategies
analytics.py	Blast radius, completion time, exposure, unnecessary delays, safety-rule violations, timelines
db.py	SQLite schema, persistence, audit log, CSV export formatting
api.py	FastAPI routes, validation and error mapping (404 / 409 / 422)
frontend/src	Eight views, the 3D fleet map (Fleet3D.tsx, instanced meshes, pointer-reactive), 2D fallback, typed API client, validation helpers
native/	C diagnostic harness, CSV exporter, GDB command script and captured session log

5.3 Database structure

Table 5 – SQLite schema

Table	Columns / content
machines, patches	id; data (JSON document of the synthetic record)
settings	key; value (scoring configuration, fleet seed and size)
rollouts	id; created_at; updated_at; strategy; scenario_id; status; state (full serialised rollout state)
experiments	id; created_at; scenario_id; seed; trials; result (summary, representative run, per-trial metrics)
audit_events	id; ts (UTC); category; action; subject; rule; detail (JSON); rollout_id; experiment_id; sim_minute

5.4 API design

Table 6 – HTTP API

Method	Path	Purpose
GET	/api/health, /api/dashboard	Status; dashboard aggregates
GET	/api/fleet, /api/fleet/{id}, /api/fleet/facets	Search, filter, sort; one machine with full explanation
POST	/api/fleet/reseed	Regenerate the synthetic fleet from a seed (audited)
GET/PUT/POST	/api/config/scoring (+ /reset)	Read, update (validated, audited), reset scoring configuration
GET	/api/patches, /api/scenarios, /api/sources	Patch catalogue; scenarios; research sources
POST	/api/rollouts, /{id}/advance, /{id}/abort	Create, step and abort a rollout (invalid transitions return 409)
GET	/api/rollouts, /api/rollouts/{id}	List; details with metrics and timelines
POST/GET	/api/experiments, /api/experiments/{id}	Run and read multi-trial experiments
GET	/api/analytics	Latest experiment per scenario
GET	/api/audit, /api/audit/export?format=csv|json	Filtered audit log; export

5.5 Technology stack (installed versions)

Table 7 – Technology stack

Layer	Technology	Version
Backend	Python / FastAPI / Starlette / Pydantic / Uvicorn	3.14.7 / 0.143.3 / 1.8.0 / 2.14.1 / 0.54.0

Layer	Technology	Version
Backend tests	pytest / httpx	9.1.1 / 0.28.1
Frontend	React / TypeScript / Vite / Tailwind CSS / Framer Motion	18.3.1 / 5.6.3 / 5.4.11 / 3.4.15 / 11.11.17
3D and charts	Three.js / @react-three/fiber / Recharts	0.169.0 / 8.17.10 / 2.13.3
Frontend tests	Vitest	2.1.5
Native demo	GCC / GDB (WSL2 Ubuntu 26.04)	15.2.0 / 17.1

All libraries are open source with permissive licences (MIT, BSD-3-Clause for Uvicorn). The code, copy and diagrams are original to this project.

6. Algorithms and safety gates

6.1 Urgency score

urgency = $100 \times \Sigma (w_i \times factor_i)$ with the four weights normalised to sum to 1. Defaults and factor definitions:

Table 8 – Urgency factors

Factor	Value in 0..1	Default weight
Severity	min(1, CVSS/10 + 0.15 if known-exploited)	0.40
Exposure	internet 1.0, internal 0.5, isolated 0.2	0.25
Criticality	(criticality − 1) / 4	0.15
Patch age	min(1, days since fix released / 60)	0.20

Bands (editable): critical ≥ 70, high ≥ 55, medium ≥ 35, otherwise low.

6.2 Compatibility risk and uncertainty

Mandatory checks are driver_compat, app_compat and backup_verified (weight 1.0 each); optional checks are agent_compat and disk_space (0.5 each); telemetry weighs 1.5. Compatibility risk = weight of checks that failed ÷ weight of all checks (excluding telemetry). Uncertainty = weight of unknown checks (plus telemetry weight if absent) ÷ total weight. A missing result is never counted as a pass.

6.3 Readiness rules and safety gates

Table 9 – Readiness rules

Code	Rule	Effect
B1	A mandatory check failed	negative evidence → blocked
B2	A mandatory check has no result	missing evidence → insufficient evidence
B3	No health telemetry (policy default: required)	missing evidence → insufficient evidence
B4	Vendor/safeguard hold for this OS version	negative evidence → blocked
B5	Pre-update latency ratio > 1.5 or error rate > 3%	negative evidence → blocked
B6	OS family needs manual approval (default ICS/Embedded)	needs approval
W1–W3	Optional check failed/unknown; telemetry optional but missing	warning → ready with caution

Precedence: blocked > insufficient evidence > needs approval > ready with caution > ready. Unsafe compatibility states (failed mandatory check, vendor hold, degraded health) and missing mandatory evidence or telemetry therefore cannot yield “ready”; they defer the machine from the staged rollout. If an operator

disables the telemetry requirement, staged rollouts still fail closed: a machine whose health cannot be verified during the soak is counted unhealthy.

6.4 Rollout state transitions

Table 10 – Rollout state machine

From	Allowed next states	Triggered by
planned	running, aborted	advance (first batch) / abort
running	running, completed, halted, aborted	advance: next batch passes / last batch passes / breach / abort
completed, halted, aborted	none (terminal)	any further advance or abort returns HTTP 409

Staged execution per batch: deploy → soak → health evaluation. A machine is unhealthy if unreachable, latency/baseline > 1.5, or error rate > 3%. The rollout halts if the canary has any unhealthy machine or if a batch's unhealthy fraction exceeds 10%. Unhealthy machines are rolled back and later batches are marked skipped and never touched. The baseline deploys to every in-scope machine at once with no gate, no soak and no halt.

6.5 Worked examples (from the running application, seed 2026)

Example A – urgent and deployable (vm-081, Windows Server 2022, internet-facing, criticality 5). Severity: CVSS 8.1 → 0.81 + 0.15 exploited bonus = 0.96 × 0.40 × 100 = 38.4. Exposure: 1.0 × 0.25 × 100 = 25.0. Criticality: (5−1)/4 = 1.0 × 0.15 × 100 = 15.0. Age: 14/60 = 0.233 × 0.20 × 100 = 4.7. Urgency = 83.1 (critical). All mandatory checks pass and telemetry is present, so readiness is Ready and the action is "deploy – expedited canary" (Figure 3).

Example B – blocked (vm-003, macOS 15, internal, criticality 2). Urgency = 47.45, displayed as 47.5 (severity 21.2 + exposure 12.5 + criticality 3.75 + age 10.0). The driver_compat check failed (rule B1), so compatibility risk = 1.0 / 4.0 = 25% (failed weight 1.0 over risk weight 3×1.0 + 2×0.5) and uncertainty = 0%. Readiness is Blocked and the action is "resolve blockers" (Figure 4).

Example C – automatic halt (rollout #9, injected bad build, seed 2026). Canary (2 machines) passed with 0 unhealthy and Stage 1 passed with 1 of 10 unhealthy (10%, which is not greater than the 10% tolerance). Stage 2 had 3 of 17 unhealthy = 17.6%, shown as 18%, which exceeds 10%, so the rollout halted; the 19 machines of Stage 3 were never touched (Figures 6–7 and the audit record in Figure 9).

6.6 Failure model (hidden ground truth of the simulation)

Independent probabilities combine as $p = 1 - \Pi(1 - p_i)$, capped at 0.97: background 0.01; failed driver check 0.55; failed application check 0.45; each unknown mandatory check 0.08; already degraded 0.30; vendor hold 0.50; plus any injected fault. A machine fails if a SHA-256-derived uniform value for (seed, machine, patch) is below p, so both strategies face identical outcomes for a given seed. 15% of non-boot failures are "subtle regressions" that remain under the default thresholds and are counted as undetected degradations.

6.7 Metrics

Table 11 – Experiment metrics

Metric	Definition
Failure blast radius	Deployed machines whose hidden outcome is failure (also reported criticality-weighted)
Completion time	Simulated minutes until the rollout stopped, including rollbacks; plus the fraction of the fleet patched
Simulated unpatched exposure	Σ (urgency/100) × hours unprotected over a 168-hour horizon

Metric	Definition
Unnecessary delays	Machines deferred by the gate or skipped after a halt whose hidden outcome would have been success
Safety-rule violations	Deployments to a machine with an unmet blocker (B1–B6), a criticality ≥ 4 machine in the canary, or a breach with no ability to halt

7. Implementation verification

7.1 Commands executed and results

Table 12 – Verification commands

Check	Command (Windows)	Result
Backend tests	cd backend; .\.venv\Scripts\python -m pytest -v	56 passed
Frontend tests	cd frontend; npm test	12 passed
Type-check and production build	cd frontend; npm run build	succeeded
Native cross-check	gcc -g -O0 -Wall -Wextra rollout_gate.c; ./rollout_gate fleet_export.csv (WSL)	120 machines, 0 mismatches

Table 13 – pytest results per file

Backend test file	Tests passed
test_experiments_api.py	22
test_fleet.py	3
test_scoring.py	14
test_simulator.py	17
Total	56

7.2 Evidence that unsafe states cannot silently pass

Table 14 – Safety properties and their tests

Property	Test (backend/tests)
Failed mandatory check is never ready	test_failed_mandatory_check_blocks_and_never_ready
Missing mandatory evidence is not treated as safe	test_missing_mandatory_evidence_is_not_silently_safe
Missing telemetry blocks	test_missing_telemetry_blocks; test_default_policy_defers_machine_without_telemetry
Fail-closed without telemetry when the requirement is relaxed	test_missing_telemetry_fails_closed_unless_disabled
Negative evidence outranks missing evidence	test_negative_evidence_outranks_missing_evidence
Canary failure halts and later batches are untouched	test_canary_failure_halts_and_skips_later_batches
Later-batch fraction above tolerance halts	test_later_batch_halts_when_fraction_exceeds_tolerance
Staged strategy has zero rule violations; baseline does not	test_baseline_ignores_gates_staged_has_zero_violations
Invalid transitions rejected	test_batch_progression_and_state_machine; test_abort_planned_rollout_is_allowed_and_terminal
Audit records exist and are not duplicated	test_rollout_lifecycle_and_audit; test_database_persistence_across_restart

7.3 Defect found and fixed

During a launcher run the log showed an intermittent HTTP 500 on /api/fleet: “SQLite objects created in a thread can only be used in that same thread”. The per-request connection was opened in one worker thread and used in another. The connection is now opened with check_same_thread=False (each request owns its connection and uses it sequentially). The regression test test_concurrent_requests_do_not_trip_sqlite_thread_affinity fails without the fix and passes with it. The algorithms were not changed.

7.4 Vitest and build logs

```
 RUN  v2.1.5 C:/Users/aniru/OneDrive/Documents/OpenMolBench/OSGuard/frontend

 ✓ src/lib/lib.test.ts (12 tests) 22ms

 Test Files  1 passed (1)
      Tests  12 passed (12)
   Start at  17:51:48
   Duration  2.48s (transform 184ms, setup 0ms, collect 190ms, tests 22ms, environment 0ms, prepare 220ms)

> osguard-frontend@1.0.0 build
> tsc --noEmit && vite build

vite v5.4.11 building for production...
transforming...
✓ 1202 modules transformed.
rendering chunks...
computing gzip size...
dist/index.html                   1.01 kB │ gzip:   0.57 kB
dist/assets/index-BcnEZxuW.css    24.72 kB │ gzip:   5.51 kB
dist/assets/Fleet3D-BRDp9SwB.js    4.78 kB │ gzip:   2.12 kB
dist/assets/index-Bto0j1sz.js    201.01 kB │ gzip:  63.01 kB
dist/assets/charts-Dbs_7qzL.js   553.11 kB │ gzip: 156.46 kB
dist/assets/three-vxG-7cUJ.js    816.09 kB │ gzip: 219.29 kB
✓ built in 12.94s
```

8. Experiments

8.1 Methodology and reproducible configuration

- Fleet: generate_fleet(seed 2026, 120 machines). Scoring: default ScoringConfig. Trials: 10 outcome seeds per scenario (2026–2035).
- Rollout defaults: batches 5/20/35/40 %, soak 30 min, deploy base 5 min + 4 min per wave of 20 machines, max latency ratio 1.5, max error rate 3%, canary strict, batch tolerance 10%, rollback 20 min, baseline manual detection 60 min, horizon 168 h, background failure probability 0.01.
- For each scenario and trial, both strategies run on the same fleet copy and the same hidden outcomes; metrics are averaged (mean) across trials.
- Reproduce: Experiment lab → Run all scenarios (seed 2026, 10 trials), or POST /api/experiments with {scenario_id, seed: 2026, trials: 10}. Results are deterministic.

Table 15 – Scenario definitions

Scenario	Fleet rewrite / overrides
Healthy update	All checks pass, telemetry on, latency at baseline, vendor holds removed; background failure probability 0
Compatibility failures	35% of Windows Server and Ubuntu hosts fail driver_compat; 20% of database/api hosts fail app_compat
Critical vulnerability	Every patch CVSS ≥ 9.8 and known-exploited; internet-facing hosts criticality ≥ 4; soak shortened to 10 min
Missing telemetry	30% of machines lose telemetry; half of those also lose app_compat; background failure probability 0.12
Degraded application health	25% of machines already at 1.8–3.0× baseline latency and 4–9% errors

Scenario	Fleet rewrite / overrides
Injected fault (bad build)	60% of Windows Server hosts hit an error-spike regression regardless of recorded evidence

8.2 Results (simulation output)

Table 16 – Means over 10 trials, shown as staged / all-at-once

Scenario	Blast radius	Safety viol.	Unneeded delays	Completion (min)	Exposure (urg·h)	Fleet patched	Staged halts
Healthy update	0.0 / 0.0	0 / 8	8.0 / 0.0	164 / 29	1039 / 18	92% / 100%	0/10
Compatibility failures	0.4 / 25.6	0 / 95	39.8 / 0.0	164 / 109	6888 / 3184	36% / 70%	0/10
Critical vulnerability	0.4 / 17.2	0 / 80	37.2 / 0.0	84 / 109	6408 / 2633	47% / 78%	0/10
Missing telemetry	1.8 / 28.7	0 / 132	63.9 / 0.0	114 / 109	9425 / 3248	10% / 69%	8/10
Degraded health	0.3 / 24.3	0 / 102	41.0 / 0.0	162 / 109	6662 / 4007	36% / 60%	0/10
Injected fault	4.6 / 30.3	0 / 80	45.4 / 0.0	157 / 109	7986 / 3521	26% / 68%	10/10

- The staged strategy has a far smaller blast radius and zero safety-rule violations in every scenario; the baseline ignores the rules by design.
- The staged strategy also has higher simulated exposure, lower fleet coverage and many unnecessary delays. This is the cost of gating. In the healthy scenario the 8 delayed machines are the industrial-system machines held by the approval gate (B6); this was checked directly (102 pending machines: 94 ready, 8 needs-approval, all ICS/Embedded).
- In the injected-fault scenario the staged blast radius is 4.6, not zero: the fault is not predictable from recorded evidence, so containment depends on the canary and the halt rule. Staged rollouts halted in 10/10 trials.

8.3 Limitations

- The failure model makes failures more likely on machines with failed or missing evidence, so evidence gating helps by construction. The injected-fault and subtle-regression cases show limits but do not remove this circularity.
- Time is a coarse simulated clock; no networks, dependencies, maintenance windows, reboot ordering or retries after rollback are modelled.
- Vendor holds, CVSS values and patch identifiers are invented; no real company incidents or telemetry are used.
- Results describe the mechanism and its trade-offs, not real-world improvement.

9. Native debugging with GDB

9.1 Applicability

GDB debugs native executables. OSGuard's application code is Python and TypeScript, so GDB does not apply to it and no claim is made that it was debugged with GDB. To give the safety rules a genuine native debugging demonstration, the team wrote a small, clearly labelled C diagnostic harness, native/rollout_gate.c. It is not part of the application, not a kernel and not xv6. It re-implements (a) the readiness gate (rules B1–B6, default configuration) and (b) the automatic-halt decision, reads the exported fleet (native/fleet_export.csv, written by native/export_fleet.py from the Python engine), recomputes every verdict and compares it with the Python result.

9.2 Build and cross-check

```
# Windows (PowerShell), from backend/ : export fleet and Python verdicts
$env:PYTHONPATH = (Get-Location).Path
.\.venv\Scripts\python ..\native\export_fleet.py ..\native\fleet_export.csv
```

```
# WSL Ubuntu : compile with debug symbols and run
gcc -g -O0 -Wall -Wextra -o /tmp/rollout_gate rollout_gate.c
/tmp/rollout_gate fleet_export.csv
# result: machines=120 mismatches=0  (ready 42, ready_with_caution 6, needs_approval 4,
#                                     insufficient_evidence 14, blocked 36, compliant 18)
```

The native counts equal the Python engine's counts for the default fleet; the compilation produced no warnings under -Wall -Wextra.

9.3 GDB session (real output, GDB 17.1, WSL2 Ubuntu 26.04)

Command: gdb -q -batch -x gdb_session.gdb --args /tmp/rollout_gate fleet_export.csv. The transcript below is the unedited captured output (native/gdb_session.log).

```
### 1. Conditional breakpoint on the readiness gate for machine vm-003
Breakpoint 1 at 0x133a: file rollout_gate.c, line 49.


This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Debuginfod has been disabled.
To make this setting permanent, add 'set debuginfod enabled off' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".


Breakpoint 1, assess_readiness (m=0x555555558190 <fleet+272>, cfg=0x7fffffffe3b0, blockers_out=0x7fffffffe39c) at rollout_gate.c:49
49    {


### 2. Backtrace and inspect the arguments
#0  assess_readiness (m=0x555555558190 <fleet+272>, cfg=0x7fffffffe3b0, blockers_out=0x7fffffffe39c) at rollout_gate.c:49
#1  0x0000555555555a5b in main (argc=2, argv=0x7fffffffe538) at rollout_gate.c:131
$1 = {
  id = "vm-003\000\000\000\000\000\000\000\000\000",
  family = "macOS", '\000' <repeats 18 times>,
  version = "15", '\000' <repeats 13 times>,
  expected = "blocked", '\000' <repeats 24 times>,
  driver = CHK_FAIL,
  app = CHK_PASS,
  backup = CHK_PASS,
  agent = CHK_PASS,
  disk = CHK_PASS,
  telemetry = 1,
  vendor_hold = 0,
  compliant = 0,
  latency_ratio = 0.99921099999999996,
  error_rate = 0.0067999999999999996
}
$2 = {
  require_telemetry = 1,
  latency_ratio_max = 1.5,
  error_rate_max = 0.029999999999999999,
  max_unhealthy_fraction = 0.10000000000000001,
  canary_halt_on_any_failure = 1
}

### 3. Watch the blocker bit mask while the rules execute
Hardware watchpoint 2: blockers


Hardware watchpoint 2: blockers


Old value = 2
New value = 0
assess_readiness (m=0x555555558190 <fleet+272>, cfg=0x7fffffffe3b0, blockers_out=0x7fffffffe39c) at rollout_gate.c:50
50        int blockers = 0, negative = 0, missing = 0, warnings = 0;
$3 = 0
$4 = 0


Hardware watchpoint 2: blockers


Old value = 0
New value = 1
0x00005555555553bd in assess_readiness (m=0x555555558190 <fleet+272>, cfg=0x7fffffffe3b0, blockers_out=0x7fffffffe39c) at
rollout_gate.c:57
57            if (mandatory[i] == CHK_FAIL)         { blockers |= B1; negative = 1; }


### 4. Finish the function and read the verdict


Watchpoint 2 deleted because the program has left the block in
which its expression is valid.
0x0000555555555a5b in main (argc=2, argv=0x7fffffffe538) at rollout_gate.c:131
131           Readiness r = assess_readiness(&fleet[i], &cfg, &blockers);
Value returned is $5 = R_BLOCKED
```

```
### 5. Break on the automatic-halt rule and test the canary boundary
Breakpoint 3 at 0x555555555505: file rollout_gate.c, line 76.


Breakpoint 3, should_halt (batch_index=0, batch_size=2, unhealthy=1, cfg=0x7fffffffe3b0) at rollout_gate.c:76
76        double frac = batch_size ? (double)unhealthy / batch_size : 0.0;
batch_index = 0
batch_size = 2
unhealthy = 1
cfg = 0x7fffffffe3b0
$6 = 0.10000000000000001
0x0000555555555baa in main (argc=2, argv=0x7fffffffe538) at rollout_gate.c:142
142        printf("halt(canary,2,1)=%d expect 1\n", should_halt(0, 2, 1, &cfg));
Value returned is $7 = 1


### 6. Evaluate the 10 percent tolerance boundary by hand
$8 = 0
$9 = 1
machines=120 mismatches=0
  ready                 42
  ready_with_caution    6
  needs_approval        4
  insufficient_evidence  14
  blocked               36
  compliant             18
halt(canary,2,1)=1 expect 1
halt(stage1,10,1)=0 expect 0 (10% is not > 10%)
halt(stage1,10,2)=1 expect 1
[Inferior 1 (process 461) exited normally]
```

9.4 What the session shows

- **Step 1–2:** a conditional breakpoint stops only on machine vm-003; the backtrace shows assess_readiness called from main, and *m shows driver = CHK_FAIL with telemetry present.
- **Step 3:** a hardware watchpoint on the blocker bit mask records its changes. The first reported change (Old value = 2 → 0) is the uninitialised stack slot being set by the declaration at line 50; the second (0 → 1) is the B1 bit being set when the failed mandatory check is processed at line 57.
- **Step 4:** finish returns R_BLOCKED, matching the Python verdict “blocked” for vm-003 (rule B1).
- **Step 5–6:** should_halt(batch 0, size 2, unhealthy 1) returns 1 (canary rule); evaluating 1/10 > 0.10 gives 0 and 2/10 > 0.10 gives 1, confirming the strict “greater than 10%” tolerance.

Limits of this evidence: it demonstrates that an independent native implementation of two rules agrees with the Python engine on this fleet and lets the logic be inspected in a debugger. It does not prove the Python code correct in general.

10. Screenshots (captured from the running application)

All screenshots were captured on 26 October 2026 from the application served at http://127.0.0.1:8000 on the Windows 11 host, after the page and its charts had finished rendering. Data are synthetic.

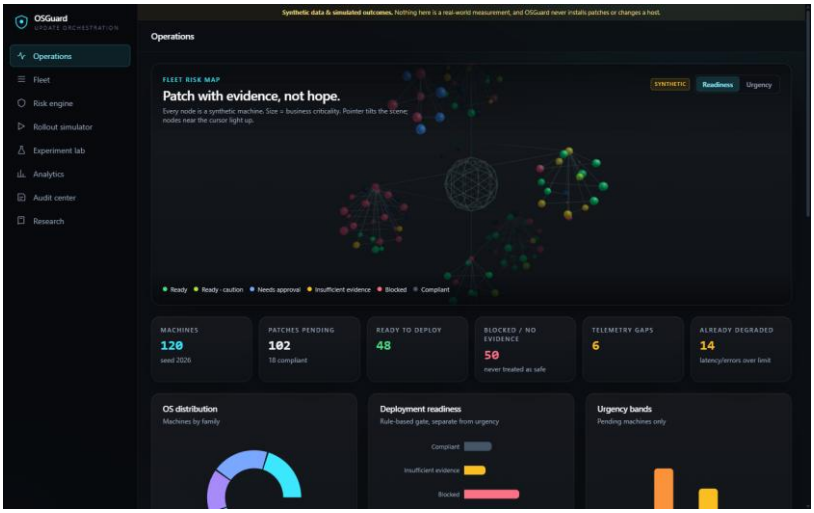


Figure 2 – Operations dashboard with the interactive 3D fleet map (nodes coloured by readiness).

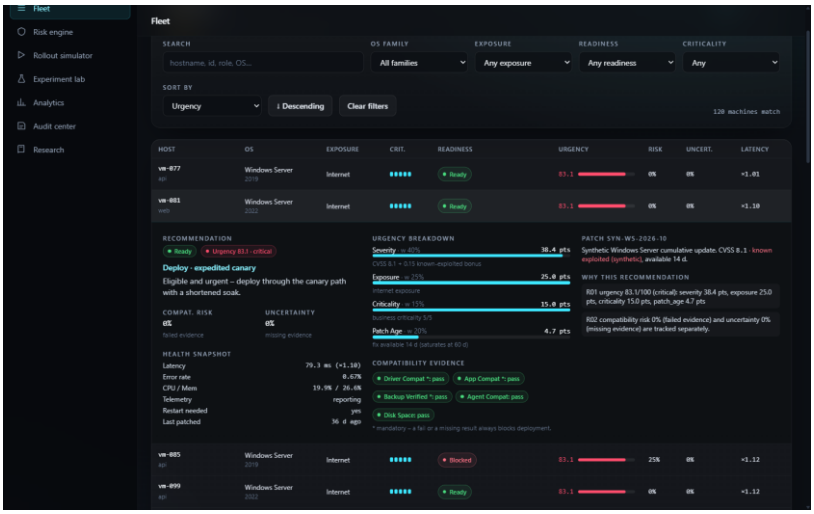


Figure 3 – Fleet inventory with search/filter/sort and an expanded machine (vm-081): recommendation, urgency breakdown, evidence, health.

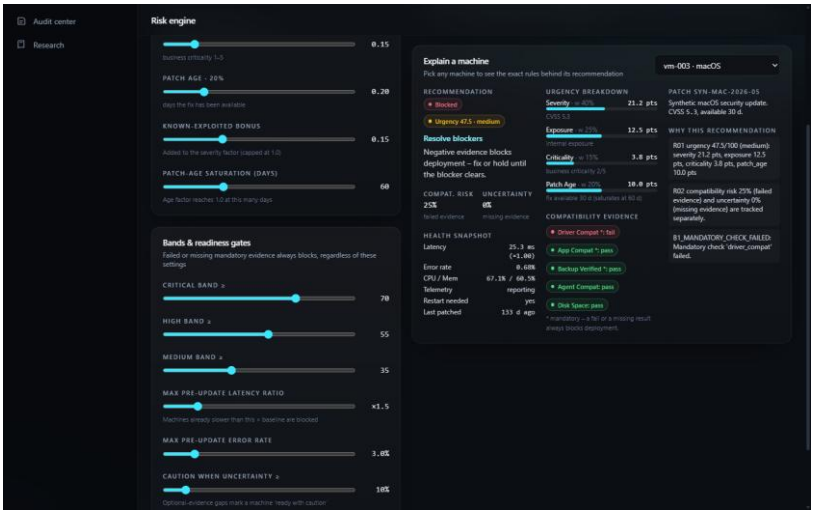


Figure 4 – Risk engine: explanation of a blocked machine (vm-003) with rule B1 and separate risk/uncertainty.

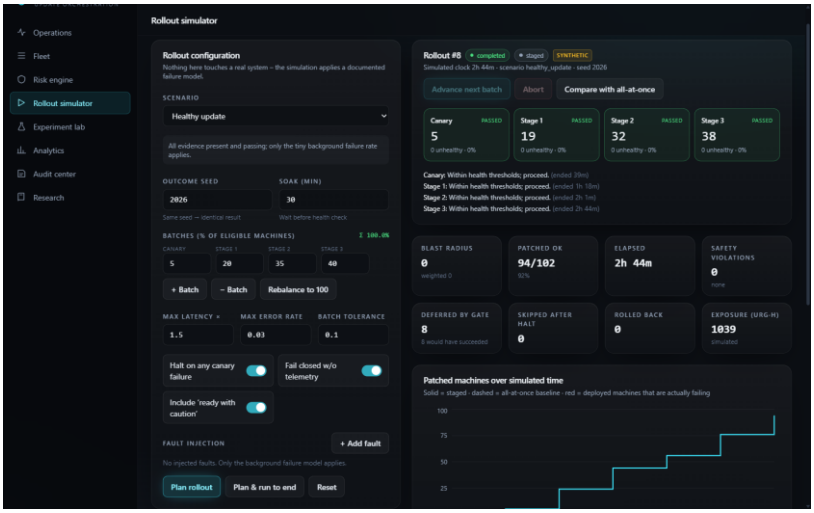


Figure 5 – Rollout simulator: successful staged rollout, healthy-update scenario (all batches passed, 0 blast radius).

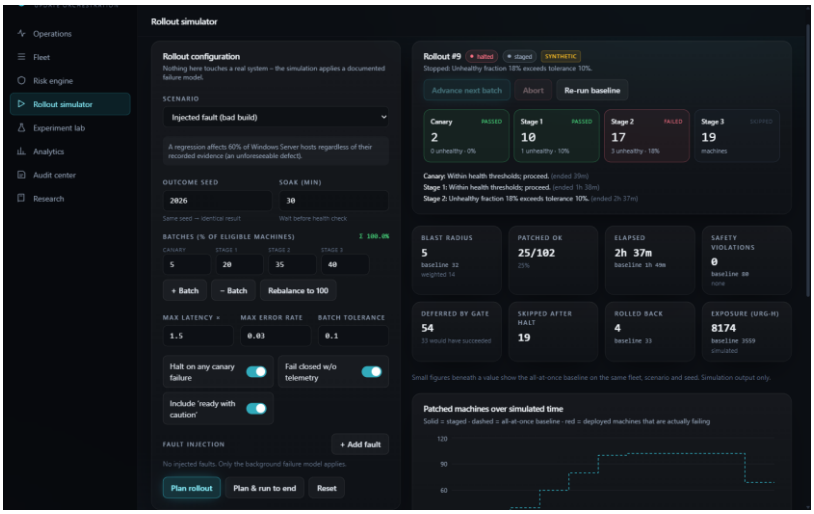


Figure 6 – Rollout simulator: injected bad build; Stage 2 fails the 10% tolerance and the rollout halts, Stage 3 untouched; baseline figures alongside.

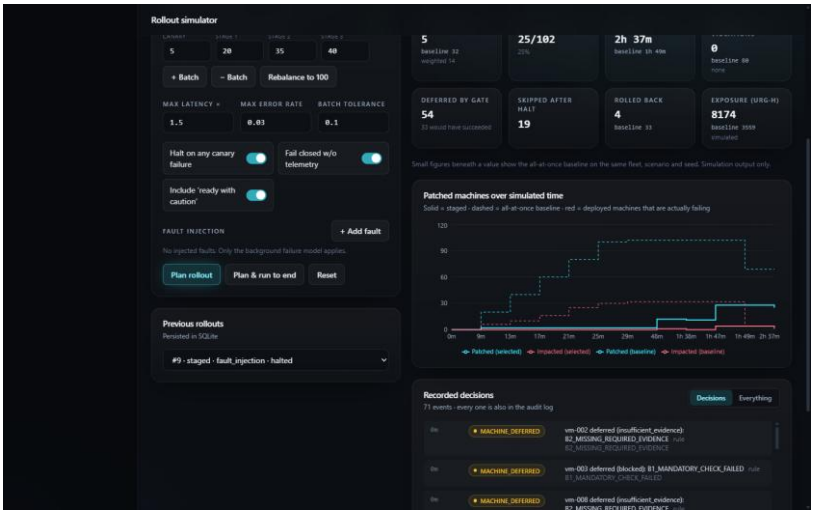


Figure 7 – Patched and impacted machines over simulated time, staged (solid) versus all-at-once baseline (dashed).



Figure 8 – Analytics: staged versus all-at-once across scenarios, with the simulation disclaimer.

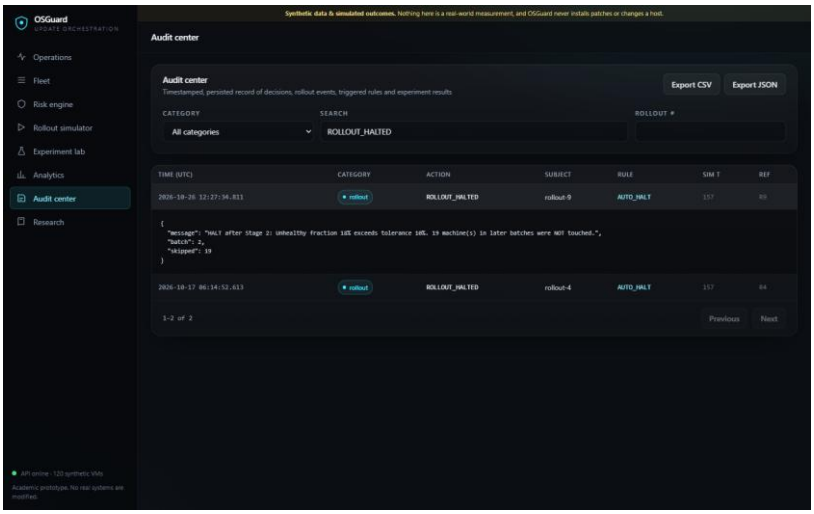


Figure 9 – Audit center: persisted ROLLOUT_HALTED event with its rule (AUTO_HALT) and details.

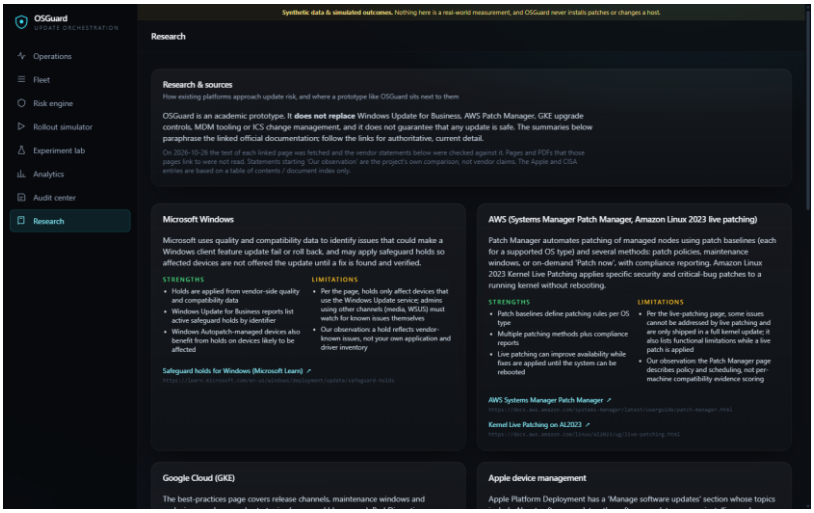


Figure 10 – Research and sources page with the verified vendor summaries.

11. Viva questions and answers

Q1. Why are urgency, compatibility risk and uncertainty separate? They answer different questions. A highly urgent machine can still be unsafe to patch, and an unknown result is not the same as a failed one. Blending them would hide why a machine was held and could let missing evidence look like safety.

Q2. What happens when a mandatory check has no result? Rule B2 makes the machine “insufficient evidence”, so it is deferred from the staged rollout. Only negative evidence (B1, B4, B5) ranks higher (blocked). Missing evidence is never rounded up to ready.

Q3. How is the canary chosen? Eligible machines are ordered by criticality (lowest first); the canary is picked round-robin across OS families from that order so it covers several platforms without risking the most critical machines.

Q4. What stops a bad update from spreading? After each batch’s soak the health check halts the rollout if the canary has any unhealthy machine or a batch’s unhealthy fraction exceeds 10%. Unhealthy machines are rolled back and later batches are never touched.

Q5. Why does the staged strategy show higher exposure? Deferred and skipped machines stay unpatched for the horizon and batches take time. Safety is bought with exposure and delay; the application reports both openly.

Q6. Is the benefit of staging proven? No. The failure model correlates failures with recorded evidence, so gating helps by construction. The results demonstrate the mechanism and its trade-offs on synthetic data only.

Q7. What is a subtle regression? A failure that stays below the default health thresholds (latency 1.2–1.4× baseline, error rate 1.5–2.5%). It is not detected and is counted as undetected degradation; tightening thresholds can catch it.

Q8. Why not machine learning? The goal is explainability and reproducibility. A rule-based model can show exactly which rule fired; no learned model exists in the project and none is claimed.

Q9. How are experiments reproducible? The fleet comes from a fixed seed and each machine's hidden outcome is a SHA-256-derived value of (seed, machine, patch), independent of Python's hash randomisation. Same inputs give identical results.

Q10. Where does GDB fit in, since the app is Python? GDB only debugs the C diagnostic harness, which re-implements two safety rules and cross-checks the Python verdicts. It is not xv6, not a kernel and not the application.

Q11. What failure did you find and fix? An intermittent HTTP 500 caused by an SQLite connection used on a different thread than the one that opened it. Fixed with check_same_thread=False plus a concurrency regression test.

Q12. How would it scale beyond 120 machines? Planning and assessment are linear in machines and run in memory; the 500-machine API limit and single-file SQLite are the first limits. A real deployment would need a job queue, per-tenant storage, authentication and real telemetry; none is implemented.

Q13. Why does this not replace Patch Manager, Windows Update or GKE tooling? Those products actually deploy and enforce updates and use vendor data. OSGuard is a simulation that never touches a host and models decision logic only.

Q14. What do the cited sources actually support? Section 4 lists exactly what each page says; Apple and CISA were checked at index level only. Our own comparisons are labelled as observations.

12. Conclusion and future work

OSGuard shows, on a reproducible synthetic fleet, how separating urgency from failed and missing evidence, gating on explicit rules, and rolling out in health-checked stages can contain a bad update at the cost of delay and exposure. The application, tests, audit trail, experiments and a native cross-check were built and verified; the findings are limited to the simulation. Future work: model dependencies and maintenance windows, ingest real (consented) telemetry, calibrate the failure model against published incident data, add authentication and a job queue, and evaluate detection of subtle regressions.

13. References

1. Microsoft Learn – Safeguard holds for Windows. https://learn.microsoft.com/en-us/windows/deployment/update/safeguard-holds (checked 26 Oct 2026).
2. AWS – AWS Systems Manager Patch Manager. https://docs.aws.amazon.com/systems-manager/latest/userguide/patch-manager.html (checked 26 Oct 2026).
3. AWS – Kernel Live Patching on AL2023. https://docs.aws.amazon.com/linux/al2023/ug/live-patching.html (checked 26 Oct 2026).
4. Google Cloud – GKE: best practices for upgrading clusters. https://docs.cloud.google.com/kubernetes-engine/docs/best-practices/upgrading-clusters (checked 26 Oct 2026).
5. Apple – Apple Platform Deployment. https://support.apple.com/guide/deployment/welcome/web (index checked 26 Oct 2026).

6. CISA – ICS Recommended Practices. https://www.cisa.gov/resources-tools/resources/ics-recommended-practices (index checked 26 Oct 2026).
7. Open-source software used: FastAPI, Pydantic, Uvicorn, pytest, React, Vite, Tailwind CSS, Framer Motion, Three.js, @react-three/fiber, Recharts, Vitest, GCC and GDB; each under its own licence. The VIT logo on the title page was supplied by the team.

14. Submission checklist and video package

Table 17 – Checklist

Item	Status
PDF contains explanation, code, GDB log, screenshots	Done (this document)
Fill in editable title-page fields (course, lecturer, slot, date)	Team action
Add the lecturer's exact assignment questions and answers, if any were set	Team action
Record the walkthrough: every member speaks in their own voice, runs the program (script: docs/VIDEO_WALKTHROUGH_SCRIPT.md)	Team action
Upload to Microsoft Clipchamp or Google Drive and share with the evaluator's address given in the assignment (manoov.r@vit.ac.in in the text supplied), or YouTube with visibility Unlisted	Team action
Paste the video link on the title page and below	Team action
Fill the speaker timestamp table below after recording	Team action
Be ready for follow-up questions on your own implementation	Team action

Video link: [EDITABLE – paste after upload]

Table 18 – Speaker allocation (segments follow docs/scripts; fill in timestamps after recording)

Team member	Suggested segment	Timestamp range
Anirudh Koppuravuri (24BCE0490)	Person 1 – introduction, OS used, architecture, dashboard and 3D map, fleet inventory	[mm:ss – mm:ss]
Kalivarapu Prabhas (24BCE2279)	Person 2 – scoring code, risk engine page, research sources, native C harness and GDB	[mm:ss – mm:ss]
Barath Sornaraj (24BCE2599)	Person 3 – simulator code and page, experiment lab, analytics, audit center, tests, conclusion	[mm:ss – mm:ss]

Appendix A. Source listings

Complete essential project logic, reproduced directly from the files. Each heading shows the file, its line count and the first 16 hex characters of its SHA-256 so the listing can be matched to the repository. Third-party code, generated files and the UI views are excluded; the UI source is in frontend/src.

A.1 scoring.py

File: backend/osguard/scoring.py | 189 lines | SHA-256 (first 16 hex): a2851da4720a3b73

Urgency computation, compatibility risk and uncertainty, readiness rules B1–B6, recommended actions and explanations.

```
"""Transparent, configurable patch-risk engine.


Three quantities are tracked *separately* and never folded into one opaque number:


* urgency      – how much we want the patch applied (severity, exposure, criticality, age)
* compat risk  – evidence of NEGATIVE compatibility results (failed checks)
* uncertainty  – MISSING evidence (unknown checks, absent telemetry)


Deployment readiness is decided by explicit rules. A failed mandatory check, a missing
mandatory check, or missing telemetry can only ever *block* a machine, never make it look safe.
"""
from __future__ import annotations


from .models import (
    MANDATORY_CHECKS,
    OPTIONAL_CHECKS,
    Assessment,
    Component,
    Finding,
    Machine,
    Patch,
    ScoringConfig,
    Urgency,
)




def _band(score: float, cfg: ScoringConfig) -> str:
    if score >= cfg.band_critical:
        return "critical"
    if score >= cfg.band_high:
        return "high"
    if score >= cfg.band_medium:
        return "medium"
    return "low"




def compute_urgency(machine: Machine, patch: Patch, cfg: ScoringConfig) -> Urgency:
    total_w = cfg.w_severity + cfg.w_exposure + cfg.w_criticality + cfg.w_age
    sev = min(1.0, patch.cvss / 10.0 + (cfg.exploited_bonus if patch.known_exploited else 0.0))
    exp = cfg.exposure_values[machine.exposure]
    crit = (machine.criticality - 1) / 4.0
    # Patch age = days the fix has been available but not applied.
    age =min(1.0, max(patch.released_days_ago, 0) / cfg.age_saturation_days)
    rows = [
        ("severity", sev, cfg.w_severity,
         f"CVSS {patch.cvss:.1f}" + (f" + {cfg.exploited_bonus:.2f} known-exploited bonus" if patch.known_exploited else "")),
        ("exposure", exp, cfg.w_exposure, f"{machine.exposure} exposure"),
        ("criticality", crit, cfg.w_criticality, f"business criticality {machine.criticality}/5"),
        ("patch_age", age, cfg.w_age,
         f"fix available {patch.released_days_ago} d (saturates at {cfg.age_saturation_days} d)"),
    ]
    comps = [
        Component(name=n, raw=round(r, 4), weight=round(w / total_w, 4), points=round(100 * r * w / total_w, 2), note=note)
        for n, r, w, note in rows
    ]
    score = round(sum(c.points for c in comps), 2)
    return Urgency(score=score, band=_band(score, cfg), components=comps)  # type: ignore[arg-type]




def compute_compat(machine: Machine, cfg: ScoringConfig) -> tuple[float, float]:
    """Return (compatibility_risk, uncertainty), each 0..1."""
    wm, wo, wt = cfg.uncertainty_weight_mandatory, cfg.uncertainty_weight_optional, cfg.uncertainty_weight_telemetry
    total = wm * len(MANDATORY_CHECKS) + wo * len(OPTIONAL_CHECKS) + wt
    failed = unknown = 0.0
    for name, status in machine.checks.items():
        w = wm if name in MANDATORY_CHECKS else wo
        if status == "fail":
            failed += w
        elif status == "unknown":
            unknown += w
    if not machine.telemetry_present:
        unknown += wt
    # Failed evidence is normalised without the telemetry term (telemetry is not a pass/fail check).
    risk_total = wm * len(MANDATORY_CHECKS) + wo * len(OPTIONAL_CHECKS)
    return round(failed / risk_total, 4), round(unknown / total, 4)




def assess_machine(machine: Machine, patch: Patch, cfg: ScoringConfig) -> Assessment:
    urgency = compute_urgency(machine, patch, cfg)
    risk, uncertainty = compute_compat(machine, cfg)
```

```
    if machine.patch_status == "compliant":
        return Assessment(
            machine_id=machine.id, urgency=urgency, compatibility_risk=risk, uncertainty=uncertainty,
            readiness="compliant", blockers=[], warnings=[], action="no_action",
            summary="Already compliant with the current patch; nothing to deploy.",
            reasons=["R00 patch_status is compliant – excluded from rollout scope."],
        )


    blockers: list[Finding] = []
    warnings: list[Finding] = []
    negative: list[str] = []  # codes that are *negative* evidence
    missing: list[str] = []  # codes that are *absent* evidence


    for name in MANDATORY_CHECKS:
        st = machine.checks.get(name, "unknown")
        if st == "fail":
            blockers.append(Finding(code="B1_MANDATORY_CHECK_FAILED", message=f"Mandatory check '{name}' failed."))
            negative.append("B1")
        elif st == "unknown":
            blockers.append(Finding(code="B2_MISSING_REQUIRED_EVIDENCE",
                                    message=f"Mandatory check '{name}' has no result; absence of evidence is treated as NOT safe."))
            missing.append("B2")
    if cfg.require_telemetry and not machine.telemetry_present:
        blockers.append(Finding(code="B3_NO_TELEMETRY",
                                message="No health telemetry: post-update health cannot be verified, so rollout is not permitted."))
        missing.append("B3")
    if machine.os_version in patch.hold_versions:
        blockers.append(Finding(code="B4_VENDOR_HOLD", message=patch.hold_reason or "Vendor safeguard hold applies."))
        negative.append("B4")
    h = machine.health
    if h.latency_ratio > cfg.health_latency_ratio_max or h.error_rate > cfg.health_error_rate_max:
        blockers.append(Finding(
            code="B5_PRE_UPDATE_HEALTH_DEGRADED",
            message=f"Machine already unhealthy (latency x{h.latency_ratio:.2f} vs limit x{cfg.health_latency_ratio_max:.2f}, "
                    f"errors {h.error_rate:.1%} vs limit {cfg.health_error_rate_max:.1%}); a regression would be unattributable."))
        negative.append("B5")
    needs_approval = machine.os_family in cfg.manual_approval_families
    if needs_approval:
        blockers.append(Finding(code="B6_MANUAL_APPROVAL_REQUIRED",
                                message=f"{machine.os_family} changes require explicit human/vendor approval."))


    for name in OPTIONAL_CHECKS:
        st = machine.checks.get(name, "unknown")
        if st == "fail":
            warnings.append(Finding(code="W1_OPTIONAL_CHECK_FAILED", message=f"Optional check '{name}' failed."))
        elif st == "unknown":
            warnings.append(Finding(code="W2_OPTIONAL_CHECK_UNKNOWN", message=f"Optional check '{name}' has no result."))
    if not cfg.require_telemetry and not machine.telemetry_present:
        warnings.append(Finding(code="W3_TELEMETRY_OPTIONAL", message="Telemetry missing but policy does not require it; staged rollouts will fail
closed."))


    soft_blockers = [b for b in blockers if b.code == "B6_MANUAL_APPROVAL_REQUIRED"]
    hard = [b for b in blockers if b not in soft_blockers]


    # Precedence: negative evidence > missing evidence > approval gate > caution > ready
    if negative:
        readiness = "blocked"
    elif missing:
        readiness = "insufficient_evidence"
    elif hard:  # pragma: no cover - defensive; all hard blockers are classified above
        readiness = "blocked"
    elif soft_blockers:
        readiness = "needs_approval"
    elif warnings or uncertainty >= cfg.caution_uncertainty:
        readiness = "ready_with_caution"
    else:
        readiness = "ready"


    critical = urgency.band == "critical"
    if readiness == "ready":
        action = "deploy_expedited" if critical else "deploy_staged"
    elif readiness == "ready_with_caution":
        action = "deploy_expedited" if critical else "deploy_staged_caution"
    elif readiness == "needs_approval":
        action = "request_approval"
    elif readiness == "insufficient_evidence":
        action = "escalate_mitigate" if critical else "collect_evidence"
    else:
        action = "escalate_mitigate" if critical else "resolve_blockers"


    reasons = [
        f"R01 urgency {urgency.score:.1f}/100 ({urgency.band}): "
        + ", ".join(f"{c.name} {c.points:.1f} pts" for c in urgency.components),
        f"R02 compatibility risk {risk:.0%} (failed evidence) and uncertainty {uncertainty:.0%} (missing evidence) are tracked separately.",
    ]
    reasons += [f"{b.code}: {b.message}" for b in blockers]
    reasons += [f"{w.code}: {w.message}" for w in warnings]


    summaries = {
        "deploy_expedited": "Eligible and urgent – deploy through the canary path with a shortened soak.",
        "deploy_staged": "Eligible – include in the normal canary → staged rollout.",
        "deploy_staged_caution": "Eligible with caveats (optional evidence missing/failed) – stage late, watch closely.",
        "request_approval": "No technical blocker, but an approval gate applies – obtain sign-off before deploying.",
        "collect_evidence": "Required evidence is missing – gather it before scheduling. Not treated as safe.",
        "resolve_blockers": "Negative evidence blocks deployment – fix or hold until the blocker clears.",
        "escalate_mitigate": "Urgent but NOT deployable – escalate to a human and apply compensating controls (isolate/restrict exposure) meanwhile.",
    }
    return Assessment(
        machine_id=machine.id, urgency=urgency, compatibility_risk=risk, uncertainty=uncertainty,
        readiness=readiness, blockers=blockers, warnings=warnings, action=action,  # type: ignore[arg-type]
        summary=summaries[action], reasons=reasons,
    )
```

```
def assess_fleet(machines: list[Machine], patches: dict[str, Patch], cfg: ScoringConfig) -> dict[str, Assessment]:
    return {m.id: assess_machine(m, patches[m.patch_id], cfg) for m in machines}



DEPLOYABLE = ("ready", "ready_with_caution")
```

A.2 simulator.py

File: backend/osguard/simulator.py | 354 lines | SHA-256 (first 16 hex): 5c1c0b80ad4c1d70

Failure model, planning (eligibility, ordering, canary), batch execution, health evaluation, halt, abort and the rollout state machine.

```
"""Rollout simulator: staged (canary → batches, health-gated) vs. all-at-once baseline.


Nothing here touches a real system. "Outcomes" are produced by a documented probabilistic
failure model keyed by a hash of (seed, machine, patch), so both strategies see exactly the
same latent truth for a given seed – only their *decisions* differ.
"""
from __future__ import annotations


import hashlib
import math


from .models import (
    BatchPlan,
    BatchResult,
    Deferred,
    FaultKind,
    Machine,
    MachineResult,
    Assessment,
    Patch,
    RolloutConfig,
    RolloutEvent,
    RolloutState,
    ScoringConfig,
)
from .scoring import DEPLOYABLE


ALLOWED_TRANSITIONS: dict[str, set[str]] = {
    "planned": {"running", "aborted"},
    "running": {"running", "completed", "halted", "aborted"},
    "completed": set(),
    "halted": set(),
    "aborted": set(),
}
TERMINAL = {"completed", "halted", "aborted"}


# Failure-model constants (documented in docs/ALGORITHMS.md).
P_DRIVER_FAIL = 0.55
P_APP_FAIL = 0.45
P_UNKNOWN_MANDATORY = 0.08
P_PRE_DEGRADED = 0.30
P_VENDOR_HOLD = 0.50
P_SUBTLE = 0.15  # fraction of non-boot failures that stay under the health thresholds



class InvalidTransition(Exception):
    pass



def unit(seed: int, *parts: object) -> float:
    """Deterministic uniform [0,1) from a hash – independent of Python's hash randomisation."""
    h = hashlib.sha256(("|".join(str(p) for p in (seed, *parts))).encode()).digest()
    return int.from_bytes(h[:8], "big") / 2**64



# ---------------------------------------------------------------------------- failure model



def failure_model(m: Machine, patch: Patch, cfg: RolloutConfig, scoring: ScoringConfig, seed: int) -> tuple[bool, FaultKind | None]:
    """Return (fails, kind) – the simulation's hidden ground truth for this machine."""
    parts: list[tuple[float, FaultKind]] = [(cfg.base_failure_probability, "latency_regression")]
    if m.checks.get("driver_compat") == "fail":
        parts.append((P_DRIVER_FAIL, "boot_failure"))
    if m.checks.get("app_compat") == "fail":
        parts.append((P_APP_FAIL, "error_spike"))
    unknown = sum(1 for c in ("driver_compat", "app_compat", "backup_verified") if m.checks.get(c) == "unknown")
    if unknown:
        parts.append((1 - (1 - P_UNKNOWN_MANDATORY) ** unknown, "latency_regression"))
    h = m.health
    if h.latency_ratio > scoring.health_latency_ratio_max or h.error_rate > scoring.health_error_rate_max:
        parts.append((P_PRE_DEGRADED, "latency_regression"))
    if m.os_version in patch.hold_versions:
        parts.append((P_VENDOR_HOLD, "boot_failure"))
    for f in cfg.faults:
        if f.matches(m):
            parts.append((f.probability, f.kind))
    survive = 1.0
    for p, _ in parts:
```

```
        survive *= 1 - min(max(p, 0.0), 1.0)
    p_fail = min(0.97, 1 - survive)
    if unit(seed, "fail", m.id, patch.id) >= p_fail:
        return False, None
    kind = max(parts, key=lambda t: t[0])[1]
    if kind != "boot_failure" and unit(seed, "subtle", m.id) < P_SUBTLE:
        kind = "subtle_regression"
    return True, kind



def post_update_health(m: Machine, fails: bool, kind: FaultKind | None, seed: int) -> tuple[float | None, float]:
    """(latency_ms or None if unreachable, error_rate) observed after the simulated update."""
    base = m.health.baseline_latency_ms
    j = lambda tag, lo, hi: lo + (hi - lo) * unit(seed, tag, m.id)  # noqa: E731
    if not fails:
        return m.health.latency_ms * j("jl", 1.0, 1.12), m.health.error_rate * j("je", 1.0, 1.2)
    if kind == "boot_failure":
        return None, 1.0
    if kind == "latency_regression":
        return base * j("jl", 2.2, 4.0), m.health.error_rate * 1.5
    if kind == "error_spike":
        return m.health.latency_ms * 1.1, j("je", 0.06, 0.2)
    return base * j("jl", 1.2, 1.4), j("je", 0.015, 0.025)  # subtle_regression



def is_unhealthy(m: Machine, latency: float | None, error: float, cfg: RolloutConfig) -> bool:
    if latency is None:
        return True
    return latency / m.health.baseline_latency_ms > cfg.max_latency_ratio or error > cfg.max_error_rate



# ---------------------------------------------------------------------------- planning



def _emit(state: RolloutState, minute: float, kind: str, message: str, *, batch: int | None = None,
          machine_id: str | None = None, rule: str | None = None, **data: object) -> RolloutEvent:
    ev = RolloutEvent(seq=len(state.events), minute=round(minute, 2), kind=kind, batch=batch,
                      machine_id=machine_id, rule=rule, message=message, data=dict(data))
    state.events.append(ev)
    return ev



def _split_sizes(n: int, percents: list[float]) -> list[int]:
    """Convert percentages to non-empty integer batch sizes summing to n."""
    if n == 0:
        return []
    total = sum(percents)
    bounds, acc = [], 0.0
    for p in percents:
        acc += p
        bounds.append(round(n * acc / total))
    bounds[-1] = n
    sizes, prev = [], 0
    for b in bounds:
        sizes.append(max(b - prev, 0))
        prev = b
    if sizes[0] == 0:  # always have a real canary
        sizes[0] = 1
        for i in range(1, len(sizes)):
            if sizes[i] > 0:
                sizes[i] -= 1
                break
    return [s for s in sizes if s > 0]



def _select_canary(ordered: list[Machine], size: int) -> list[Machine]:
    """Round-robin across OS families so the canary covers as many platforms as possible."""
    groups: dict[str, list[Machine]] = {}
    for m in ordered:
        groups.setdefault(m.os_family, []).append(m)
    chosen: list[Machine] = []
    while len(chosen) < size and any(groups.values()):
        for fam in sorted(groups):
            if groups[fam] and len(chosen) < size:
                chosen.append(groups[fam].pop(0))
    return chosen



def plan_rollout(
    machines: list[Machine],
    patches: dict[str, Patch],
    assessments: dict[str, Assessment],
    cfg: RolloutConfig,
    scoring: ScoringConfig,
    *,
    scenario_id: str = "none",
    seed: int = 2026,
) -> RolloutState:
    state = RolloutState(strategy=cfg.strategy, scenario_id=scenario_id, seed=seed, config=cfg)
    scope = [m for m in machines if m.patch_status == "pending"]
    for m in scope:
        fails, kind = failure_model(m, patches[m.patch_id], cfg, scoring, seed)
        state.results[m.id] = MachineResult(machine_id=m.id, failed=fails, fault_kind=kind, would_succeed=not fails)


    if cfg.strategy == "all_at_once":
        ids = sorted(m.id for m in scope)
        state.batches = [BatchPlan(index=0, label="All at once", machine_ids=ids)] if ids else []
```

```
        _emit(state, 0, "PLAN", f"Baseline: deploy to all {len(ids)} in-scope machines simultaneously. "
              "No readiness gate, no health gate, no halt.", rule="BASELINE")
        return state


    eligible: list[Machine] = []
    for m in scope:
        a = assessments[m.id]
        ok = a.readiness in DEPLOYABLE and (cfg.allow_caution or a.readiness == "ready")
        if ok:
            eligible.append(m)
            continue
        codes = list(dict.fromkeys(b.code for b in a.blockers)) or ["POLICY_NO_CAUTION"]
        state.deferred.append(Deferred(machine_id=m.id, readiness=a.readiness, codes=codes))
        state.results[m.id].state = "deferred"
        _emit(state, 0, "MACHINE_DEFERRED", f"{m.id} deferred ({a.readiness}): {'; '.join(codes)}",
              machine_id=m.id, rule=codes[0], readiness=a.readiness, codes=codes)


    ordered = sorted(eligible, key=lambda m: (m.criticality, assessments[m.id].readiness != "ready", -assessments[m.id].urgency.score, m.id))
    sizes = _split_sizes(len(ordered), cfg.batch_percents)
    if sizes:
        canary = _select_canary(ordered, sizes[0])
        chosen = {m.id for m in canary}
        rest = [m for m in ordered if m.id not in chosen]
        groups = [canary]
        pos = 0
        for s in sizes[1:]:
            groups.append(rest[pos:pos + s])
            pos += s
        for i, g in enumerate(groups):
            state.batches.append(BatchPlan(index=i, label="Canary" if i == 0 else f"Stage {i}", machine_ids=[m.id for m in g]))
    _emit(state, 0, "PLAN",
          f"Staged plan: {len(ordered)} eligible machines in {len(state.batches)} batches {sizes}; "
          f"{len(state.deferred)} deferred by readiness gate.", rule="STAGED_PLAN",
          eligible=len(ordered), deferred=len(state.deferred), sizes=sizes)
    return state



# ---------------------------------------------------------------------------- execution



def advance(
    state: RolloutState,
    machines: dict[str, Machine],
    patches: dict[str, Patch],
    scoring: ScoringConfig,
) -> list[RolloutEvent]:
    """Run the next batch to completion (deploy → soak → health evaluation → decision)."""
    if "running" not in ALLOWED_TRANSITIONS[state.status]:
        raise InvalidTransition(f"cannot advance a rollout in state '{state.status}'")
    first = len(state.events)
    cfg = state.config
    if state.next_batch >= len(state.batches):
        state.status = "completed"
        _emit(state, state.clock, "ROLLOUT_COMPLETED", "Nothing to deploy; rollout closed.", rule="EMPTY")
        return state.events[first:]
    state.status = "running"
    batch = state.batches[state.next_batch]
    start = state.clock
    n = len(batch.machine_ids)
    _emit(state, start, "BATCH_STARTED", f"{batch.label}: deploying to {n} machine(s).", batch=batch.index, rule="BATCH_START", size=n)


    deploy_end = start
    for k, mid in enumerate(batch.machine_ids):
        t = start + cfg.deploy_base_minutes + math.ceil((k + 1) / cfg.concurrency) * cfg.minutes_per_wave
        r = state.results[mid]
        r.state, r.patched_at = "patched", t
        deploy_end = max(deploy_end, t)
        _emit(state, t, "MACHINE_PATCHED", f"{mid} updated (simulated).", batch=batch.index, machine_id=mid)


    unhealthy: list[str] = []
    unverifiable: list[str] = []
    detected_failing: list[str] = []
    for mid in batch.machine_ids:
        m, r = machines[mid], state.results[mid]
        lat, err = post_update_health(m, r.failed, r.fault_kind, state.seed)
        if cfg.strategy == "staged" and not m.telemetry_present:
            unverifiable.append(mid)  # cannot verify health: fail closed unless explicitly disabled
            if cfg.fail_closed_on_missing_telemetry:
                unhealthy.append(mid)
            continue
        if is_unhealthy(m, lat, err, cfg):
            unhealthy.append(mid)
            detected_failing.append(mid)


    if cfg.strategy == "all_at_once":
        # Baseline has no gate: failures surface later through manual detection, then rollback.
        end = deploy_end
        for mid in detected_failing:
            r = state.results[mid]
            det = deploy_end + cfg.baseline_detect_minutes
            r.unhealthy_detected, r.state, r.rolled_back_at = True, "rolled_back", det + cfg.rollback_minutes
            end = max(end, r.rolled_back_at)
            _emit(state, det, "FAILURE_DETECTED", f"{mid} failure noticed manually after {cfg.baseline_detect_minutes} min ({r.fault_kind}).",
                  batch=0, machine_id=mid, rule="MANUAL_DETECTION", fault=r.fault_kind)
            _emit(state, r.rolled_back_at, "MACHINE_ROLLED_BACK", f"{mid} rolled back.", batch=0, machine_id=mid)
        for mid in batch.machine_ids:
            r = state.results[mid]
            if r.failed and r.state == "patched":
                r.state = "undetected_degraded"
        frac = len(detected_failing) / n if n else 0.0
        breach = frac > cfg.max_unhealthy_fraction
        state.breach_seen = breach
        state.batch_results.append(BatchResult(index=0, label=batch.label, start_minute=start, end_minute=end, size=n,
```

```
                                            unhealthy=len(detected_failing), unverifiable=len(unverifiable),
                                            unhealthy_fraction=round(frac, 4), passed=not breach,
                                            reason="No halt mechanism: failures handled manually after the fact." if breach else "Within tolerance."))
        _emit(state, end, "ROLLOUT_COMPLETED", f"Baseline wave finished with {len(detected_failing)} failures "
              f"({frac:.0%}); rollout could not be halted.", rule="NO_HALT", unhealthy_fraction=round(frac, 4))
        state.clock, state.next_batch, state.status = end, 1, "completed"
        return state.events[first:]


    eval_at = deploy_end + cfg.soak_minutes
    _emit(state, deploy_end, "SOAK_STARTED", f"{batch.label}: soaking {cfg.soak_minutes} min before health evaluation.", batch=batch.index, rule="SOAK")
    frac = len(unhealthy) / n if n else 0.0
    canary_hit = batch.index == 0 and cfg.canary_halt_on_any_failure and len(unhealthy) > 0
    breach = canary_hit or frac > cfg.max_unhealthy_fraction
    _emit(state, eval_at, "HEALTH_CHECK",
          f"{batch.label}: {len(unhealthy)}/{n} unhealthy ({frac:.0%}), {len(unverifiable)} unverifiable; "
          f"limits: latency x{cfg.max_latency_ratio}, errors {cfg.max_error_rate:.1%}, batch tolerance {cfg.max_unhealthy_fraction:.0%}.",
          batch=batch.index, rule="HEALTH_THRESHOLDS", unhealthy=len(unhealthy), unverifiable=len(unverifiable), fraction=round(frac, 4))


    end = eval_at
    for mid in unhealthy:
        r = state.results[mid]
        r.unhealthy_detected, r.state, r.rolled_back_at = True, "rolled_back", eval_at + cfg.rollback_minutes
        end = max(end, r.rolled_back_at)
        why = "telemetry missing – failing closed" if mid in unverifiable else f"health threshold exceeded ({r.fault_kind})"
        _emit(state, eval_at, "MACHINE_ROLLED_BACK", f"{mid} rolled back: {why}.", batch=batch.index, machine_id=mid,
              rule="FAIL_CLOSED" if mid in unverifiable else "HEALTH_THRESHOLDS")
    for mid in batch.machine_ids:
        r = state.results[mid]
        if r.failed and r.state == "patched":
            r.state = "undetected_degraded"


    if breach:
        reason = (f"Canary failure detected ({len(unhealthy)}/{n} unhealthy)." if canary_hit
                  else f"Unhealthy fraction {frac:.0%} exceeds tolerance {cfg.max_unhealthy_fraction:.0%}.")
        state.batch_results.append(BatchResult(index=batch.index, label=batch.label, start_minute=start, end_minute=end, size=n,
                                               unhealthy=len(unhealthy), unverifiable=len(unverifiable),
                                               unhealthy_fraction=round(frac, 4), passed=False, reason=reason))
        remaining = 0
        for b in state.batches[batch.index + 1:]:
            for mid in b.machine_ids:
                state.results[mid].state = "skipped"
                remaining += 1
        state.status, state.halt_reason, state.breach_seen = "halted", reason, True
        state.clock, state.next_batch = end, batch.index + 1
        _emit(state, end, "ROLLOUT_HALTED", f"HALT after {batch.label}: {reason} {remaining} machine(s) in later batches were NOT touched.",
              batch=batch.index, rule="AUTO_HALT", skipped=remaining)
        return state.events[first:]


    state.batch_results.append(BatchResult(index=batch.index, label=batch.label, start_minute=start, end_minute=end, size=n,
                                           unhealthy=len(unhealthy), unverifiable=len(unverifiable),
                                           unhealthy_fraction=round(frac, 4), passed=True, reason="Within health thresholds; proceed."))
    _emit(state, end, "BATCH_PASSED", f"{batch.label} passed; promoting to next batch." if batch.index + 1 < len(state.batches)
          else f"{batch.label} passed.", batch=batch.index, rule="BATCH_PASS")
    state.clock, state.next_batch = end, batch.index + 1
    if state.next_batch >= len(state.batches):
        state.status = "completed"
        _emit(state, end, "ROLLOUT_COMPLETED", "All batches completed.", rule="COMPLETE")
    return state.events[first:]



def abort(state: RolloutState, reason: str = "operator abort") -> list[RolloutEvent]:
    if "aborted" not in ALLOWED_TRANSITIONS[state.status]:
        raise InvalidTransition(f"cannot abort a rollout in state '{state.status}'")
    first = len(state.events)
    for b in state.batches[state.next_batch:]:
        for mid in b.machine_ids:
            state.results[mid].state = "skipped"
    state.status, state.halt_reason = "aborted", reason
    _emit(state, state.clock, "ROLLOUT_ABORTED", f"Rollout aborted: {reason}.", rule="OPERATOR_ABORT")
    return state.events[first:]



def run_to_end(state: RolloutState, machines: dict[str, Machine], patches: dict[str, Patch], scoring: ScoringConfig) -> RolloutState:
    while state.status not in TERMINAL:
        advance(state, machines, patches, scoring)
    return state
```

A.3 analytics.py

File: backend/osguard/analytics.py | 136 lines | SHA-256 (first 16 hex): eee7d138fd05afdc

Safety-rule violations, exposure curve, timelines and experiment metrics.

```
"""Comparison metrics computed from finished (or halted) simulated rollouts.


Every number produced here is a *simulation output*. It is measured on the synthetic fleet
under the documented failure model and says nothing about real-world outcomes.
"""
from __future__ import annotations


from statistics import mean


from .models import Assessment, Machine, RolloutState


DEPLOYED_STATES = {"patched", "rolled_back", "undetected_degraded"}
```

```
def violations(state: RolloutState, machines: dict[str, Machine], assessments: dict[str, Assessment]) -> list[dict]:
    """Safety-rule violations: deploying where the readiness rules said not to."""
    out: list[dict] = []
    for mid, r in state.results.items():
        if r.patched_at is None:
            continue
        a = assessments[mid]
        for b in a.blockers:
            out.append({"machine_id": mid, "rule": b.code, "message": f"Deployed despite: {b.message}"})
        if state.strategy == "staged" and state.batches and mid in state.batches[0].machine_ids and machines[mid].criticality >= 4:
            out.append({"machine_id": mid, "rule": "V7_CRITICAL_IN_CANARY",
                        "message": f"Criticality {machines[mid].criticality} machine placed in the canary batch."})
    if state.strategy == "all_at_once" and state.breach_seen:
        out.append({"machine_id": "*", "rule": "V8_NO_HALT_ON_BREACH",
                    "message": "Health thresholds were breached but the rollout had no mechanism to halt."})
    return out



def _unprotected_hours(r, horizon_min: float) -> list[tuple[float, float]]:
    """Intervals (minutes) during which the machine is not protected by the patch."""
    if r.state in ("patched", "undetected_degraded") and r.patched_at is not None:
        return [(0.0, min(r.patched_at, horizon_min))]
    if r.state == "rolled_back" and r.patched_at is not None and r.rolled_back_at is not None:
        return [(0.0, min(r.patched_at, horizon_min)), (min(r.rolled_back_at, horizon_min), horizon_min)]
    return [(0.0, horizon_min)]



def exposure_curve(state: RolloutState, assessments: dict[str, Assessment], points: int = 28) -> list[dict]:
    horizon = state.config.horizon_hours * 60.0
    res = []
    for k in range(points + 1):
        t = horizon * k / points
        total = 0.0
        for mid, r in state.results.items():
            w = assessments[mid].urgency.score / 100.0
            total += w * sum(max(0.0, min(b, t) - a) for a, b in _unprotected_hours(r, horizon)) / 60.0
        res.append({"hour": round(t / 60.0, 2), "exposure": round(total, 2)})
    return res



def timeline(state: RolloutState) -> list[dict]:
    """Step series: patched / rolled back / degraded-but-live machines over simulated minutes."""
    marks = {0.0}
    for r in state.results.values():
        for t in (r.patched_at, r.rolled_back_at):
            if t is not None:
                marks.add(t)
    series = []
    for t in sorted(marks):
        patched = rolled = impacted = 0
        for r in state.results.values():
            if r.patched_at is not None and r.patched_at <= t:
                if r.rolled_back_at is not None and r.rolled_back_at <= t:
                    rolled += 1
                else:
                    patched += 1
                    if r.failed:
                        impacted += 1
        series.append({"minute": round(t, 2), "patched": patched, "rolled_back": rolled, "impacted": impacted})
    return series



def compute_metrics(state: RolloutState, machines: dict[str, Machine], assessments: dict[str, Assessment]) -> dict:
    cfg = state.config
    horizon = cfg.horizon_hours * 60.0
    scope = list(state.results.values())
    deployed = [r for r in scope if r.patched_at is not None]
    failed_deployed = [r for r in deployed if r.failed]
    blast = len(failed_deployed)
    blast_weighted = sum(machines[r.machine_id].criticality for r in failed_deployed)
    patched_ok = [r for r in scope if r.state in ("patched", "undetected_degraded")]
    would_succeed_unpatched = [r for r in scope if r.state in ("deferred", "skipped") and r.would_succeed]
    exposure = 0.0
    for r in scope:
        w = assessments[r.machine_id].urgency.score / 100.0
        exposure += w * sum(b - a for a, b in _unprotected_hours(r, horizon)) / 60.0
    viol = violations(state, machines, assessments)
    rule_counts: dict[str, int] = {}
    for v in viol:
        rule_counts[v["rule"]] = rule_counts.get(v["rule"], 0) + 1
    gate_delayed = [r for r in scope if r.state == "deferred" and r.would_succeed]
    halt_delayed = [r for r in scope if r.state == "skipped" and r.would_succeed]
    return {
        "strategy": state.strategy,
        "status": state.status,
        "in_scope": len(scope),
        "deployed": len(deployed),
        "deferred_by_gate": sum(1 for r in scope if r.state == "deferred"),
        "skipped_after_halt": sum(1 for r in scope if r.state == "skipped"),
        "patched_successfully": len(patched_ok),
        "completion_fraction": round(len(patched_ok) / len(scope), 4) if scope else 0.0,
        "completion_minutes": round(state.clock, 1),
        "blast_radius": blast,
        "blast_radius_weighted": blast_weighted,
        "blast_radius_fraction": round(blast / len(deployed), 4) if deployed else 0.0,
        "rolled_back": sum(1 for r in scope if r.state == "rolled_back"),
        "undetected_degraded": sum(1 for r in scope if r.state == "undetected_degraded"),
        "unpatched_exposure": round(exposure, 2),
        "unnecessary_delays": len(gate_delayed) + len(halt_delayed),
        "unnecessary_delays_gate": len(gate_delayed),
        "unnecessary_delays_halt": len(halt_delayed),
        "unnecessary_delay_hours": round(len(would_succeed_unpatched) * cfg.horizon_hours, 1),
        "safety_violations": len(viol),
        "safety_violation_rules": rule_counts,
```

```
        "halted": state.status == "halted",
        "halt_reason": state.halt_reason,
    }



METRIC_KEYS = [
    "blast_radius", "blast_radius_weighted", "completion_minutes", "completion_fraction", "unpatched_exposure",
    "unnecessary_delays", "safety_violations", "rolled_back", "undetected_degraded", "deployed",
]



def summarise_trials(per_trial: list[dict]) -> dict:
    out = {}
    for k in METRIC_KEYS:
        vals = [t[k] for t in per_trial]
        out[k] = {"mean": round(mean(vals), 3), "min": min(vals), "max": max(vals)}
    out["halted_trials"] = sum(1 for t in per_trial if t["halted"])
    return out
```

A.4 scenarios.py

File: backend/osguard/scenarios.py | 124 lines | SHA-256 (first 16 hex): cb351bd504267195

The six reproducible scenarios and the scenario/config builder.

```
"""Reproducible experiment scenarios.


A scenario deterministically rewrites a copy of the synthetic fleet (never the stored
inventory) and/or adjusts the failure model, so a given (scenario, seed) always replays
identically.
"""
from __future__ import annotations


import random
from dataclasses import dataclass, field
from typing import Callable


from .models import FaultInjection, Machine, Patch, RolloutConfig, Scenario



@dataclass
class ScenarioSpec:
    meta: Scenario
    mutate: Callable[[list[Machine], dict[str, Patch], random.Random], None] = lambda m, p, r: None
    config_patch: dict = field(default_factory=dict)
    faults: list[FaultInjection] = field(default_factory=list)



def _healthy(machines, patches, rng):
    for m in machines:  # a clean world: all evidence present and passing
        m.checks = {k: "pass" for k in m.checks}
        m.telemetry_present = True
        h = m.health
        h.latency_ms, h.error_rate = h.baseline_latency_ms, min(h.error_rate, 0.01)
    for p in patches.values():
        p.hold_versions = []



def _compat_failures(machines, patches, rng):
    for m in machines:
        if m.os_family in ("Windows Server", "Ubuntu") and rng.random() < 0.35:
            m.checks["driver_compat"] = "fail"
        if m.role in ("database", "api") and rng.random() < 0.20:
            m.checks["app_compat"] = "fail"



def _critical_vuln(machines, patches, rng):
    for p in patches.values():
        p.cvss, p.known_exploited = max(p.cvss, 9.8), True
    for m in machines:
        if m.exposure == "internet":
            m.criticality = max(m.criticality, 4)



def _missing_telemetry(machines, patches, rng):
    for m in machines:
        if rng.random() < 0.30:
            m.telemetry_present = False
            if rng.random() < 0.5:
                m.checks["app_compat"] = "unknown"



def _degraded(machines, patches, rng):
    for m in machines:
        if rng.random() < 0.25:
            m.health.latency_ms = round(m.health.baseline_latency_ms * rng.uniform(1.8, 3.0), 2)
            m.health.error_rate = round(rng.uniform(0.04, 0.09), 4)



SCENARIOS: dict[str, ScenarioSpec] = {
```

```
    s.meta.id: s
    for s in [
        ScenarioSpec(
            Scenario(id="healthy_update", name="Healthy update",
                     description="All evidence present and passing; only the tiny background failure rate applies.",
                     expectation="Both strategies should complete. The staged strategy pays a time cost (soak windows) and may leave exposure slightly
higher."),
            mutate=_healthy, config_patch={"base_failure_probability": 0.0},
        ),
        ScenarioSpec(
            Scenario(id="compat_failure", name="Compatibility failures",
                     description="A large share of Windows Server/Ubuntu hosts fail the driver check and many database/API hosts fail the application
check.",
                     expectation="The readiness gate should defer machines with failed evidence; the all-at-once baseline deploys to them and suffers a
large blast radius."),
            mutate=_compat_failures,
        ),
        ScenarioSpec(
            Scenario(id="critical_vuln", name="Critical vulnerability",
                     description="Every patch is rated 9.8 and known-exploited, and internet-facing hosts become highly critical. Staged soak is shortened
to 10 minutes.",
                     expectation="Shows the speed/safety trade-off: exposure matters more, but gating must still hold. Blocked critical machines escalate
instead of being deployed."),
            mutate=_critical_vuln, config_patch={"soak_minutes": 10},
        ),
        ScenarioSpec(
            Scenario(id="missing_telemetry", name="Missing telemetry",
                     description="30% of machines stop reporting health telemetry; half also lose their application-compatibility result.",
                     expectation="Absence of evidence is treated as not-safe: those machines are deferred (possibly unnecessarily). The baseline deploys
blind."),
            mutate=_missing_telemetry, config_patch={"base_failure_probability": 0.12},
        ),
        ScenarioSpec(
            Scenario(id="degraded_health", name="Degraded application health",
                     description="25% of machines are already slow/erroring before the update, and a regression there is more likely.",
                     expectation="The pre-update health gate defers degraded hosts. The baseline deploys onto them and cannot attribute regressions."),
            mutate=_degraded,
        ),
        ScenarioSpec(
            Scenario(id="fault_injection", name="Injected fault (bad build)",
                     description="A regression affects 60% of Windows Server hosts regardless of their recorded evidence (an unforeseeable defect).",
                     expectation="Evidence gating cannot predict this; the canary + automatic halt should limit the blast radius. Undetected subtle
regressions remain possible."),
            faults=[FaultInjection(kind="error_spike", probability=0.6, os_family="Windows Server", label="bad cumulative update")],
        ),
    ]
}



def apply_scenario(scenario_id: str, machines: list[Machine], patches: dict[str, Patch], seed: int):
    """Return (machines_copy, patches_copy, extra_config_patch, faults)."""
    spec = SCENARIOS.get(scenario_id)
    ms = [m.model_copy(deep=True) for m in machines]
    ps = {k: v.model_copy(deep=True) for k, v in patches.items()}
    if spec is None:
        return ms, ps, {}, []
    spec.mutate(ms, ps, random.Random(f"{scenario_id}:{seed}"))
    return ms, ps, dict(spec.config_patch), [f.model_copy() for f in spec.faults]



def build_config(base: RolloutConfig, strategy: str, patch: dict, faults: list[FaultInjection]) -> RolloutConfig:
    data = base.model_dump()
    data.update(patch)
    data["strategy"] = strategy
    data["faults"] = [f.model_dump() for f in list(base.faults) + faults]
    return RolloutConfig.model_validate(data)
```

A.5 experiments.py

File: backend/osguard/experiments.py | 55 lines | SHA-256 (first 16 hex): 49e6d419140d62cc

Multi-trial paired comparison of both strategies.

```
"""Experiment runner: same scenario, same fleet, both strategies, several outcome seeds."""
from __future__ import annotations


from .analytics import compute_metrics, exposure_curve, summarise_trials, timeline
from .models import Machine, Patch, RolloutConfig, ScoringConfig
from .scenarios import apply_scenario, build_config
from .scoring import assess_fleet
from .simulator import plan_rollout, run_to_end


MAX_TRIALS = 200



def run_experiment(
    machines: list[Machine],
    patches: dict[str, Patch],
    scoring: ScoringConfig,
    base_cfg: RolloutConfig,
    *,
    scenario_id: str,
    seed: int,
    trials: int,
) -> dict:
    if not 1 <= trials <= MAX_TRIALS:
        raise ValueError(f"trials must be within 1..{MAX_TRIALS}")
    ms, ps, cfg_patch, faults = apply_scenario(scenario_id, machines, patches, seed)
    assessments = assess_fleet(ms, ps, scoring)
    by_id = {m.id: m for m in ms}
    per: dict[str, list[dict]] = {"staged": [], "all_at_once": []}
    representative: dict[str, dict] = {}
    for t in range(trials):
```

```
        for strategy in ("staged", "all_at_once"):
            cfg = build_config(base_cfg, strategy, cfg_patch, faults)
            st = plan_rollout(ms, ps, assessments, cfg, scoring, scenario_id=scenario_id, seed=seed + t)
            run_to_end(st, by_id, ps, scoring)
            metrics = compute_metrics(st, by_id, assessments)
            per[strategy].append(metrics)
            if t == 0:
                representative[strategy] = {
                    "metrics": metrics,
                    "timeline": timeline(st),
                    "exposure_curve": exposure_curve(st, assessments),
                    "batch_results": [b.model_dump() for b in st.batch_results],
                    "events": [e.model_dump() for e in st.events if e.kind != "MACHINE_PATCHED"][:400],
                    "halt_reason": st.halt_reason,
                }
    summary = {s: summarise_trials(v) for s, v in per.items()}
    return {
        "scenario_id": scenario_id,
        "seed": seed,
        "trials": trials,
        "summary": summary,
        "representative": representative,
        "per_trial": per,
        "disclaimer": "Simulation output on a synthetic fleet; not a real-world measurement.",
    }
```

A.6 db.py

File: backend/osguard/db.py | 219 lines | SHA-256 (first 16 hex): 208afb955f761aed

SQLite schema, persistence, audit log, query and CSV formatting.

```
"""SQLite persistence: inventory, settings, rollouts, experiments and the audit log."""
from __future__ import annotations


import csv
import io
import json
import os
import sqlite3
from contextlib import contextmanager
from datetime import datetime, timezone
from pathlib import Path
from typing import Iterator


from .fleet import DEFAULT_FLEET_SIZE, DEFAULT_SEED, PATCHES, generate_fleet
from .models import Machine, Patch, RolloutState, ScoringConfig


DEFAULT_DB = Path(__file__).resolve().parent.parent / "data" / "osguard.db"


SCHEMA = """
CREATE TABLE IF NOT EXISTS machines (
  id TEXT PRIMARY KEY, data TEXT NOT NULL
);
CREATE TABLE IF NOT EXISTS patches (
  id TEXT PRIMARY KEY, data TEXT NOT NULL
);
CREATE TABLE IF NOT EXISTS settings (
  key TEXT PRIMARY KEY, value TEXT NOT NULL
);
CREATE TABLE IF NOT EXISTS rollouts (
  id INTEGER PRIMARY KEY AUTOINCREMENT, created_at TEXT NOT NULL, updated_at TEXT NOT NULL,
  strategy TEXT NOT NULL, scenario_id TEXT NOT NULL, status TEXT NOT NULL, state TEXT NOT NULL
);
CREATE TABLE IF NOT EXISTS experiments (
  id INTEGER PRIMARY KEY AUTOINCREMENT, created_at TEXT NOT NULL, scenario_id TEXT NOT NULL,
  seed INTEGER NOT NULL, trials INTEGER NOT NULL, result TEXT NOT NULL
);
CREATE TABLE IF NOT EXISTS audit_events (
  id INTEGER PRIMARY KEY AUTOINCREMENT, ts TEXT NOT NULL, category TEXT NOT NULL, action TEXT NOT NULL,
  subject TEXT, rule TEXT, detail TEXT NOT NULL DEFAULT '{}', rollout_id INTEGER, experiment_id INTEGER,
  sim_minute REAL
);
CREATE INDEX IF NOT EXISTS idx_audit_category ON audit_events(category);
CREATE INDEX IF NOT EXISTS idx_audit_rollout ON audit_events(rollout_id);
"""




def now_iso() -> str:
    return datetime.now(timezone.utc).isoformat(timespec="milliseconds")




def resolve_db_path(path: str | os.PathLike | None = None) -> str:
    p = Path(path or os.environ.get("OSGUARD_DB") or DEFAULT_DB)
    p.parent.mkdir(parents=True, exist_ok=True)
    return str(p)




@contextmanager
def connect(path: str) -> Iterator[sqlite3.Connection]:
    # FastAPI may open the dependency in one worker thread and run the handler in another; each request
    # owns its connection and uses it sequentially, so cross-thread hand-off is safe.
    conn = sqlite3.connect(path, check_same_thread=False)
    conn.row_factory = sqlite3.Row
    conn.execute("PRAGMA journal_mode=WAL")
    conn.execute("PRAGMA foreign_keys=ON")
    try:
        yield conn
        conn.commit()
    except Exception:
        conn.rollback()
```

```
        raise
    finally:
        conn.close()



def init_db(path: str) -> None:
    with connect(path) as c:
        c.executescript(SCHEMA)
        if c.execute("SELECT COUNT(*) FROM machines").fetchone()[0] == 0:
            seed_fleet(c, DEFAULT_SEED, DEFAULT_FLEET_SIZE)
            log(c, "system", "FLEET_SEEDED", subject="fleet", detail={"seed": DEFAULT_SEED, "size": DEFAULT_FLEET_SIZE, "synthetic": True})



def seed_fleet(c: sqlite3.Connection, seed: int, size: int) -> None:
    c.execute("DELETE FROM machines")
    c.execute("DELETE FROM patches")
    c.executemany("INSERT INTO patches(id,data) VALUES(?,?)", [(p.id, p.model_dump_json()) for p in PATCHES])
    c.executemany("INSERT INTO machines(id,data) VALUES(?,?)", [(m.id, m.model_dump_json()) for m in generate_fleet(seed, size)])
    c.execute("INSERT OR REPLACE INTO settings(key,value) VALUES('fleet_seed',?)", (json.dumps({"seed": seed, "size": size}),))



def load_machines(c: sqlite3.Connection) -> list[Machine]:
    return [Machine.model_validate_json(r["data"]) for r in c.execute("SELECT data FROM machines ORDER BY id")]



def load_patches(c: sqlite3.Connection) -> dict[str, Patch]:
    return {p.id: p for p in (Patch.model_validate_json(r["data"]) for r in c.execute("SELECT data FROM patches"))}



def get_scoring(c: sqlite3.Connection) -> ScoringConfig:
    row = c.execute("SELECT value FROM settings WHERE key='scoring'").fetchone()
    return ScoringConfig.model_validate_json(row["value"]) if row else ScoringConfig()



def set_scoring(c: sqlite3.Connection, cfg: ScoringConfig) -> None:
    c.execute("INSERT OR REPLACE INTO settings(key,value) VALUES('scoring',?)", (cfg.model_dump_json(),))



def get_fleet_meta(c: sqlite3.Connection) -> dict:
    row = c.execute("SELECT value FROM settings WHERE key='fleet_seed'").fetchone()
    return json.loads(row["value"]) if row else {"seed": DEFAULT_SEED, "size": DEFAULT_FLEET_SIZE}



# ---------------------------------------------------------------------------- audit



def log(c: sqlite3.Connection, category: str, action: str, *, subject: str | None = None, rule: str | None = None,
        detail: dict | None = None, rollout_id: int | None = None, experiment_id: int | None = None,
        sim_minute: float | None = None) -> int:
    cur = c.execute(
        "INSERT INTO audit_events(ts,category,action,subject,rule,detail,rollout_id,experiment_id,sim_minute) VALUES(?,?,?,?,?,?,?,?,?)",
        (now_iso(), category, action, subject, rule, json.dumps(detail or {}, default=str), rollout_id, experiment_id, sim_minute),
    )
    return int(cur.lastrowid)



def query_audit(c: sqlite3.Connection, *, category: str | None = None, q: str | None = None, rollout_id: int | None = None,
                experiment_id: int | None = None, rule: str | None = None, limit: int = 200, offset: int = 0) -> tuple[int, list[dict]]:
    where, args = [], []
    if category:
        where.append("category=?"); args.append(category)
    if rollout_id is not None:
        where.append("rollout_id=?"); args.append(rollout_id)
    if experiment_id is not None:
        where.append("experiment_id=?"); args.append(experiment_id)
    if rule:
        where.append("rule=?"); args.append(rule)
    if q:
        where.append("(action LIKE ? OR subject LIKE ? OR rule LIKE ? OR detail LIKE ?)")
        args += [f"%{q}%"] * 4
    clause = (" WHERE " + " AND ".join(where)) if where else ""
    total = c.execute(f"SELECT COUNT(*) FROM audit_events{clause}", args).fetchone()[0]
    rows = c.execute(f"SELECT * FROM audit_events{clause} ORDER BY id DESC LIMIT ? OFFSET ?", [*args, limit, offset]).fetchall()
    out = []
    for r in rows:
        d = dict(r)
        d["detail"] = json.loads(d["detail"])
        out.append(d)
    return total, out



def audit_csv(rows: list[dict]) -> str:
    buf = io.StringIO()
    w = csv.writer(buf)
    w.writerow(["id", "ts", "category", "action", "subject", "rule", "rollout_id", "experiment_id", "sim_minute", "detail"])
    for r in rows:
        w.writerow([r["id"], r["ts"], r["category"], r["action"], r["subject"], r["rule"], r["rollout_id"],
                    r["experiment_id"], r["sim_minute"], json.dumps(r["detail"])])
    return buf.getvalue()
```

```
# ---------------------------------------------------------------------------- rollouts / experiments



def save_rollout(c: sqlite3.Connection, state: RolloutState) -> RolloutState:
    """Insert or update a rollout and write any not-yet-audited events to the audit log."""
    ts = now_iso()
    if state.id is None:
        cur = c.execute("INSERT INTO rollouts(created_at,updated_at,strategy,scenario_id,status,state) VALUES(?,?,?,?,?,?)",
                        (ts, ts, state.strategy, state.scenario_id, state.status, "{}"))
        state.id = int(cur.lastrowid)
        log(c, "rollout", "ROLLOUT_CREATED", subject=f"rollout-{state.id}", rule=state.strategy.upper(), rollout_id=state.id,
            detail={"scenario": state.scenario_id, "seed": state.seed, "config": state.config.model_dump()})
    for ev in state.events[state.persisted_events:]:
        if ev.kind == "MACHINE_PATCHED":
            continue  # high-volume and implied by BATCH_STARTED; the full log stays in the rollout state
        log(c, "rollout", ev.kind, subject=ev.machine_id or f"rollout-{state.id}", rule=ev.rule, rollout_id=state.id,
            sim_minute=ev.minute, detail={"message": ev.message, "batch": ev.batch, **ev.data})
    state.persisted_events = len(state.events)
    c.execute("UPDATE rollouts SET updated_at=?, status=?, state=? WHERE id=?", (ts, state.status, state.model_dump_json(), state.id))
    return state



def load_rollout(c: sqlite3.Connection, rollout_id: int) -> RolloutState | None:
    row = c.execute("SELECT state FROM rollouts WHERE id=?", (rollout_id,)).fetchone()
    if not row:
        return None
    st = RolloutState.model_validate_json(row["state"])
    st.id = rollout_id
    return st



def list_rollouts(c: sqlite3.Connection, limit: int = 50) -> list[dict]:
    rows = c.execute("SELECT id,created_at,updated_at,strategy,scenario_id,status FROM rollouts ORDER BY id DESC LIMIT ?", (limit,)).fetchall()
    return [dict(r) for r in rows]



def save_experiment(c: sqlite3.Connection, result: dict) -> int:
    cur = c.execute("INSERT INTO experiments(created_at,scenario_id,seed,trials,result) VALUES(?,?,?,?,?)",
                    (now_iso(), result["scenario_id"], result["seed"], result["trials"], json.dumps(result)))
    return int(cur.lastrowid)



def load_experiment(c: sqlite3.Connection, exp_id: int) -> dict | None:
    row = c.execute("SELECT id,created_at,result FROM experiments WHERE id=?", (exp_id,)).fetchone()
    if not row:
        return None
    d = json.loads(row["result"])
    d["id"], d["created_at"] = row["id"], row["created_at"]
    return d



def list_experiments(c: sqlite3.Connection, limit: int = 100) -> list[dict]:
    rows = c.execute("SELECT id,created_at,scenario_id,seed,trials,result FROM experiments ORDER BY id DESC LIMIT ?", (limit,)).fetchall()
    out = []
    for r in rows:
        res = json.loads(r["result"])
        out.append({"id": r["id"], "created_at": r["created_at"], "scenario_id": r["scenario_id"], "seed": r["seed"],
                    "trials": r["trials"], "summary": res["summary"]})
    return out
```

A.7 api.py

File: backend/osguard/api.py | 322 lines | SHA-256 (first 16 hex): b6d70e2f49d722a8

FastAPI routes, validation and error mapping.

```
"""FastAPI application. Every endpoint is a thin wrapper over the pure engine modules."""
from __future__ import annotations


import json
import os
from collections import Counter
from pathlib import Path
from typing import Literal


from fastapi import Depends, FastAPI, HTTPException, Query
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import FileResponse, PlainTextResponse
from pydantic import BaseModel, Field


from . import db
from .experiments import run_experiment
from .fleet import DEFAULT_FLEET_SIZE, DEFAULT_SEED
from .models import RolloutConfig, ScoringConfig
from .scenarios import SCENARIOS, apply_scenario, build_config
from .scoring import assess_fleet, assess_machine
from .simulator import InvalidTransition, abort, advance, plan_rollout, run_to_end
from .sources import LIBRARIES, SOURCES, VERIFICATION


DISCLAIMER = ("All machines, patches and outcomes are SYNTHETIC. Results are simulation outputs, "
              "not real-world measurements. OSGuard never modifies a host or installs patches.")
DIST = Path(__file__).resolve().parent.parent.parent / "frontend" / "dist"
```

```
class RolloutRequest(BaseModel):
    config: RolloutConfig = Field(default_factory=RolloutConfig)
    scenario_id: str = "none"
    seed: int = DEFAULT_SEED
    run: bool = False  # run all batches immediately



class ExperimentRequest(BaseModel):
    scenario_id: str
    seed: int = DEFAULT_SEED
    trials: int = Field(10, ge=1, le=200)
    config: RolloutConfig = Field(default_factory=RolloutConfig)



class ReseedRequest(BaseModel):
    seed: int = Field(DEFAULT_SEED, ge=0, le=2**31 - 1)
    size: int = Field(DEFAULT_FLEET_SIZE, ge=100, le=500)



def create_app(db_path: str | None = None) -> FastAPI:
    path = db.resolve_db_path(db_path)
    db.init_db(path)
    app = FastAPI(title="OSGuard API", version="1.0.0", description=DISCLAIMER)
    app.add_middleware(CORSMiddleware, allow_origins=["http://localhost:5173", "http://127.0.0.1:5173"],
                       allow_methods=["*"], allow_headers=["*"])


    def conn():
        with db.connect(path) as c:
            yield c


    def world(c, scenario_id: str = "none", seed: int = DEFAULT_SEED):
        scoring = db.get_scoring(c)
        ms, ps = db.load_machines(c), db.load_patches(c)
        if scenario_id != "none":
            if scenario_id not in SCENARIOS:
                raise HTTPException(404, f"unknown scenario '{scenario_id}'")
            ms, ps, _, _ = apply_scenario(scenario_id, ms, ps, seed)
        return ms, ps, scoring, assess_fleet(ms, ps, scoring)


    @app.get("/api/health")
    def health(c=Depends(conn)):
        n = c.execute("SELECT COUNT(*) FROM machines").fetchone()[0]
        return {"status": "ok", "machines": n, "database": os.path.basename(path), "disclaimer": DISCLAIMER}


    # ---------------------------------------------------------------------- dashboard
    @app.get("/api/dashboard")
    def dashboard(c=Depends(conn)):
        ms, ps, _, asm = world(c)
        pending = [m for m in ms if m.patch_status == "pending"]
        by_patch = Counter(m.patch_id for m in pending)
        queue = sorted((m for m in pending if asm[m.id].readiness in ("ready", "ready_with_caution")),
                       key=lambda m: -asm[m.id].urgency.score)[:8]
        stuck = sorted((m for m in pending if asm[m.id].readiness in ("blocked", "insufficient_evidence")),
                       key=lambda m: -asm[m.id].urgency.score)[:6]
        degraded = sum(1 for m in ms if m.health.latency_ratio > 1.5 or m.health.error_rate > 0.03)
        _, recent = db.query_audit(c, limit=8)
        return {
            "meta": db.get_fleet_meta(c),
            "totals": {"machines": len(ms), "pending": len(pending), "compliant": len(ms) - len(pending),
                       "restart_required": sum(1 for m in pending if m.restart_required),
                       "no_telemetry": sum(1 for m in ms if not m.telemetry_present), "degraded": degraded},
            "os_distribution": [{"name": k, "value": v} for k, v in Counter(m.os_family for m in ms).most_common()],
            "readiness": [{"name": k, "value": v} for k, v in Counter(asm[m.id].readiness for m in ms).items()],
            "risk_bands": [{"name": b, "value": sum(1 for m in pending if asm[m.id].urgency.band == b)} for b in ("critical", "high", "medium", "low")],
            "outstanding_patches": [{"patch_id": pid, "title": ps[pid].title, "cvss": ps[pid].cvss, "known_exploited": ps[pid].known_exploited,
                                     "machines": n} for pid, n in by_patch.most_common()],
            "priority_queue": [{"machine_id": m.id, "hostname": m.hostname, "urgency": asm[m.id].urgency.score,
                                "band": asm[m.id].urgency.band, "action": asm[m.id].action} for m in queue],
            "stuck_urgent": [{"machine_id": m.id, "hostname": m.hostname, "urgency": asm[m.id].urgency.score,
                              "band": asm[m.id].urgency.band, "readiness": asm[m.id].readiness,
                              "blockers": [b.code for b in asm[m.id].blockers]} for m in stuck],
            "recent_activity": recent,
            "disclaimer": DISCLAIMER,
        }


    # ---------------------------------------------------------------------- fleet
    @app.get("/api/fleet")
    def fleet(
        search: str | None = None, os_family: str | None = None, exposure: str | None = None,
        readiness: str | None = None, criticality: int | None = Query(None, ge=1, le=5),
        patch_status: str | None = None,
        sort: Literal["urgency", "criticality", "hostname", "uncertainty", "risk", "latency"] = "urgency",
        order: Literal["asc", "desc"] = "desc", limit: int = Query(200, ge=1, le=500), offset: int = Query(0, ge=0),
        c=Depends(conn),
    ):
        ms, _, _, asm = world(c)
        items = ms
        if search:
            s = search.lower()
            items = [m for m in items if s in m.hostname.lower() or s in m.id.lower() or s in m.role.lower() or s in m.os_family.lower()]
        if os_family:
            items = [m for m in items if m.os_family == os_family]
        if exposure:
            items = [m for m in items if m.exposure == exposure]
        if criticality:
            items = [m for m in items if m.criticality == criticality]
        if patch_status:
```

```
        items = [m for m in items if m.patch_status == patch_status]
    if readiness:
        items = [m for m in items if asm[m.id].readiness == readiness]
    keyf = {
        "urgency": lambda m: asm[m.id].urgency.score, "criticality": lambda m: m.criticality,
        "hostname": lambda m: m.hostname, "uncertainty": lambda m: asm[m.id].uncertainty,
        "risk": lambda m: asm[m.id].compatibility_risk, "latency": lambda m: m.health.latency_ratio,
    }[sort]
    items = sorted(items, key=keyf, reverse=(order == "desc"))
    total = len(items)
    page = items[offset: offset + limit]
    return {"total": total, "items": [
        {**m.model_dump(), "urgency": asm[m.id].urgency.score, "band": asm[m.id].urgency.band, "readiness": asm[m.id].readiness,
         "action": asm[m.id].action, "compatibility_risk": asm[m.id].compatibility_risk, "uncertainty": asm[m.id].uncertainty,
         "blocker_codes": [b.code for b in asm[m.id].blockers]}
        for m in page]}


@app.get("/api/fleet/facets")
def facets(c=Depends(conn)):
    ms = db.load_machines(c)
    return {"os_family": sorted({m.os_family for m in ms}), "exposure": ["internet", "internal", "isolated"],
            "readiness": ["ready", "ready_with_caution", "needs_approval", "insufficient_evidence", "blocked", "compliant"]}


@app.get("/api/fleet/{machine_id}")
def machine_detail(machine_id: str, c=Depends(conn)):
    ms, ps, scoring, _ = world(c)
    m = next((x for x in ms if x.id == machine_id), None)
    if not m:
        raise HTTPException(404, f"machine '{machine_id}' not found")
    return {"machine": m, "patch": ps[m.patch_id], "assessment": assess_machine(m, ps[m.patch_id], scoring)}


@app.post("/api/fleet/reseed")
def reseed(req: ReseedRequest, c=Depends(conn)):
    db.seed_fleet(c, req.seed, req.size)
    db.log(c, "system", "FLEET_RESEEDED", subject="fleet", detail={"seed": req.seed, "size": req.size, "synthetic": True})
    return {"seed": req.seed, "size": req.size}


@app.get("/api/patches")
def patches(c=Depends(conn)):
    return list(db.load_patches(c).values())


# ---------------------------------------------------------------- config
@app.get("/api/config/scoring")
def get_scoring(c=Depends(conn)):
    return db.get_scoring(c)


@app.put("/api/config/scoring")
def put_scoring(cfg: ScoringConfig, c=Depends(conn)):
    before = db.get_scoring(c).model_dump()
    db.set_scoring(c, cfg)
    changed = {k: {"from": before[k], "to": v} for k, v in cfg.model_dump().items() if before[k] != v}
    db.log(c, "config", "SCORING_CONFIG_UPDATED", subject="scoring", rule="CONFIG", detail={"changed": changed})
    return cfg


@app.post("/api/config/scoring/reset")
def reset_scoring(c=Depends(conn)):
    cfg = ScoringConfig()
    db.set_scoring(c, cfg)
    db.log(c, "config", "SCORING_CONFIG_RESET", subject="scoring", rule="CONFIG")
    return cfg


@app.get("/api/config/rollout-defaults")
def rollout_defaults():
    return RolloutConfig()


@app.get("/api/scenarios")
def scenarios():
    return [s.meta for s in SCENARIOS.values()]


# ---------------------------------------------------------------- rollouts
@app.post("/api/rollouts", status_code=201)
def create_rollout(req: RolloutRequest, c=Depends(conn)):
    ms, ps, scoring, asm = world(c, req.scenario_id, req.seed)
    _, _, cfg_patch, faults = apply_scenario(req.scenario_id, [], {}, req.seed) if req.scenario_id != "none" else ([], {}, {}, [])
    cfg = build_config(req.config, req.config.strategy, cfg_patch, faults)
    state = plan_rollout(ms, ps, asm, cfg, scoring, scenario_id=req.scenario_id, seed=req.seed)
    if req.run:
        run_to_end(state, {m.id: m for m in ms}, ps, scoring)
    db.save_rollout(c, state)
    return state


@app.get("/api/rollouts")
def rollouts(c=Depends(conn)):
    return db.list_rollouts(c)


def get_state(c, rid: int):
    st = db.load_rollout(c, rid)
    if not st:
        raise HTTPException(404, f"rollout {rid} not found")
    return st


@app.get("/api/rollouts/{rid}")
def rollout(rid: int, c=Depends(conn)):
    from .analytics import compute_metrics, exposure_curve, timeline
    st = get_state(c, rid)
    ms, ps, _, asm = world(c, st.scenario_id, st.seed)
    by = {m.id: m for m in ms}
    return {"state": st, "metrics": compute_metrics(st, by, asm), "timeline": timeline(st), "exposure_curve": exposure_curve(st, asm)}
```

```
@app.post("/api/rollouts/{rid}/advance")
def rollout_advance(rid: int, c=Depends(conn)):
    st = get_state(c, rid)
    ms, ps, scoring, _ = world(c, st.scenario_id, st.seed)
    try:
        advance(st, {m.id: m for m in ms}, ps, scoring)
    except InvalidTransition as e:
        raise HTTPException(409, str(e))
    db.save_rollout(c, st)
    return rollout(rid, c)


@app.post("/api/rollouts/{rid}/abort")
def rollout_abort(rid: int, c=Depends(conn)):
    st = get_state(c, rid)
    try:
        abort(st)
    except InvalidTransition as e:
        raise HTTPException(409, str(e))
    db.save_rollout(c, st)
    return rollout(rid, c)


# ---------------------------------------------------------------------- experiments
@app.post("/api/experiments", status_code=201)
def create_experiment(req: ExperimentRequest, c=Depends(conn)):
    if req.scenario_id not in SCENARIOS:
        raise HTTPException(404, f"unknown scenario '{req.scenario_id}'")
    ms, ps, scoring = db.load_machines(c), db.load_patches(c), db.get_scoring(c)
    result = run_experiment(ms, ps, scoring, req.config, scenario_id=req.scenario_id, seed=req.seed, trials=req.trials)
    eid = db.save_experiment(c, result)
    s = result["summary"]
    db.log(c, "experiment", "EXPERIMENT_COMPLETED", subject=req.scenario_id, rule="EXPERIMENT", experiment_id=eid,
           detail={"seed": req.seed, "trials": req.trials, "summary": s, "synthetic": True})
    rep = result["representative"]
    for strat, data in rep.items():
        for rule, n in data["metrics"]["safety_violation_rules"].items():
            db.log(c, "experiment", "SAFETY_RULE_TRIGGERED", subject=f"{req.scenario_id}/{strat}", rule=rule, experiment_id=eid,
                   detail={"count": n, "strategy": strat})
        if data["halt_reason"]:
            db.log(c, "experiment", "AUTO_HALT", subject=f"{req.scenario_id}/{strat}", rule="AUTO_HALT", experiment_id=eid,
                   detail={"reason": data["halt_reason"]})
    result["id"] = eid
    return result


@app.get("/api/experiments")
def experiments(c=Depends(conn)):
    return db.list_experiments(c)


@app.get("/api/experiments/{eid}")
def experiment(eid: int, c=Depends(conn)):
    r = db.load_experiment(c, eid)
    if not r:
        raise HTTPException(404, f"experiment {eid} not found")
    return r


@app.get("/api/analytics")
def analytics(c=Depends(conn)):
    """Latest experiment per scenario, for side-by-side comparison."""
    latest: dict[str, dict] = {}
    for e in db.list_experiments(c, limit=500):
        latest.setdefault(e["scenario_id"], e)
    return {"scenarios": [s.meta for s in SCENARIOS.values()], "latest": list(latest.values()), "disclaimer": DISCLAIMER}


# ---------------------------------------------------------------------- audit
@app.get("/api/audit")
def audit(category: str | None = None, q: str | None = None, rule: str | None = None, rollout_id: int | None = None,
          experiment_id: int | None = None, limit: int = Query(100, ge=1, le=1000), offset: int = Query(0, ge=0), c=Depends(conn)):
    total, rows = db.query_audit(c, category=category, q=q, rule=rule, rollout_id=rollout_id, experiment_id=experiment_id,
                                 limit=limit, offset=offset)
    cats = [r["category"] for r in c.execute("SELECT DISTINCT category FROM audit_events").fetchall()]
    return {"total": total, "items": rows, "categories": sorted(cats)}


@app.get("/api/audit/export")
def audit_export(format: Literal["csv", "json"] = "csv", category: str | None = None, q: str | None = None,
                 rollout_id: int | None = None, experiment_id: int | None = None, c=Depends(conn)):
    _, rows = db.query_audit(c, category=category, q=q, rollout_id=rollout_id, experiment_id=experiment_id, limit=10000)
    if format == "json":
        return PlainTextResponse(json.dumps({"disclaimer": DISCLAIMER, "events": rows}, indent=2), media_type="application/json",
                                 headers={"Content-Disposition": "attachment; filename=osguard-audit.json"})
    return PlainTextResponse(db.audit_csv(rows), media_type="text/csv",
                             headers={"Content-Disposition": "attachment; filename=osguard-audit.csv"})


# ---------------------------------------------------------------------- research
@app.get("/api/sources")
def sources():
    return {"sources": SOURCES, "libraries": LIBRARIES, "verification": VERIFICATION}


# ---------------------------------------------------------------------- static frontend
if DIST.exists():
    @app.get("/{full_path:path}", include_in_schema=False)
    def spa(full_path: str):
        if full_path.startswith("api/"):
            raise HTTPException(404, "not found")
        f = (DIST / full_path).resolve()
        if full_path and DIST.resolve() in f.parents and f.is_file():
            return FileResponse(f)
        return FileResponse(DIST / "index.html")


return app
```

A.8 native/rollout_gate.c

File: native/rollout_gate.c | 146 lines | SHA-256 (first 16 hex): 4776296e337a3a46

C diagnostic harness: readiness gate, halt rule and CSV cross-check (debugged in GDB).

```
/*
 * rollout_gate.c - DIAGNOSTIC HARNESS (not part of the OSGuard application, not a kernel, not xv6).
 *
 * A small native re-implementation of two OSGuard safety rules so they can be inspected with GDB:
 *   1. readiness gate  (backend/osguard/scoring.py::assess_machine, rules B1-B6, default configuration)
 *   2. automatic halt  (backend/osguard/simulator.py::advance, canary + unhealthy-fraction rule)
 * It reads native/fleet_export.csv, recomputes every machine's readiness and compares it with the
 * verdict exported from the Python engine. Exit status 0 = identical, 1 = mismatch.
 *
 * Build: gcc -g -O0 -Wall -Wextra -o rollout_gate rollout_gate.c
 * Run:   ./rollout_gate fleet_export.csv
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>


typedef enum { CHK_PASS, CHK_FAIL, CHK_UNKNOWN } Check;
typedef enum { R_READY, R_CAUTION, R_NEEDS_APPROVAL, R_INSUFFICIENT, R_BLOCKED, R_COMPLIANT } Readiness;


static const char *READINESS_NAME[] = { "ready", "ready_with_caution", "needs_approval", "insufficient_evidence", "blocked", "compliant" };


/* Blocker bit mask, same codes as the Python engine. */
enum { B1 = 1, B2 = 2, B3 = 4, B4 = 8, B5 = 16, B6 = 32 };


typedef struct {
    char id[16], family[24], version[16], expected[32];
    Check driver, app, backup, agent, disk;
    int telemetry, vendor_hold, compliant;
    double latency_ratio, error_rate;
} Machine;


typedef struct {
    int require_telemetry;
    double latency_ratio_max, error_rate_max;
    double max_unhealthy_fraction;
    int canary_halt_on_any_failure;
} Config;


static const Config DEFAULTS = { 1, 1.5, 0.03, 0.10, 1 };


static Check parse_check(const char *s)
{
    if (!strcmp(s, "pass")) return CHK_PASS;
    if (!strcmp(s, "fail")) return CHK_FAIL;
    return CHK_UNKNOWN;
}


Readiness assess_readiness(const Machine *m, const Config *cfg, int *blockers_out)
{
    int blockers = 0, negative = 0, missing = 0, warnings = 0;
    const Check mandatory[3] = { m->driver, m->app, m->backup };
    int i;


    if (m->compliant) { *blockers_out = 0; return R_COMPLIANT; }


    for (i = 0; i < 3; i++) {
        if (mandatory[i] == CHK_FAIL)         { blockers |= B1; negative = 1; }
        else if (mandatory[i] == CHK_UNKNOWN) { blockers |= B2; missing = 1; }
    }
    if (cfg->require_telemetry && !m->telemetry) { blockers |= B3; missing = 1; }
    if (m->vendor_hold)                          { blockers |= B4; negative = 1; }
    if (m->latency_ratio > cfg->latency_ratio_max || m->error_rate > cfg->error_rate_max) { blockers |= B5; negative = 1; }
    if (!strcmp(m->family, "ICS/Embedded"))      { blockers |= B6; }
    if (m->agent != CHK_PASS || m->disk != CHK_PASS) warnings = 1;


    *blockers_out = blockers;
    if (negative) return R_BLOCKED;                  /* negative evidence beats everything */
    if (missing)  return R_INSUFFICIENT;             /* missing evidence is never "safe" */
    if (blockers & B6) return R_NEEDS_APPROVAL;
    return warnings ? R_CAUTION : R_READY;
}


/* Returns 1 when the rollout must halt after evaluating a batch. */
int should_halt(int batch_index, int batch_size, int unhealthy, const Config *cfg)
{
    double frac = batch_size ? (double)unhealthy / batch_size : 0.0;
    int canary_hit = (batch_index == 0 && cfg->canary_halt_on_any_failure && unhealthy > 0);
    return canary_hit || frac > cfg->max_unhealthy_fraction;
}


static int load(const char *path, Machine *out, int max)
{
    FILE *f = fopen(path, "r");
    char line[512];
    int n = 0;
    if (!f) { perror(path); return -1; }
    if (!fgets(line, sizeof line, f)) { fclose(f); return -1; }   /* header row */
    while (n < max && fgets(line, sizeof line, f)) {
        Machine *m = &out[n];
        char *tok[14], *p = line;
        int k = 0;
        while (k < 14 && p) {
```

```
            tok[k++] = p;
            p = strchr(p, ',');
            if (p) *p++ = '\0';
        }
        if (k < 14) continue;
        tok[13][strcspn(tok[13], "\r\n")] = '\0';
        snprintf(m->id, sizeof m->id, "%s", tok[0]);
        snprintf(m->family, sizeof m->family, "%s", tok[1]);
        snprintf(m->version, sizeof m->version, "%s", tok[2]);
        m->driver = parse_check(tok[3]);
        m->app = parse_check(tok[4]);
        m->backup = parse_check(tok[5]);
        m->agent = parse_check(tok[6]);
        m->disk = parse_check(tok[7]);
        m->telemetry = atoi(tok[8]);
        m->latency_ratio = atof(tok[9]);
        m->error_rate = atof(tok[10]);
        m->vendor_hold = atoi(tok[11]);
        m->compliant = atoi(tok[12]);
        snprintf(m->expected, sizeof m->expected, "%s", tok[13]);
        n++;
    }
    fclose(f);
    return n;
}


int main(int argc, char **argv)
{
    static Machine fleet[512];
    int n, i, mismatches = 0, counts[6] = { 0 };
    Config cfg = DEFAULTS;


    if (argc < 2) { fprintf(stderr, "usage: %s fleet_export.csv\n", argv[0]); return 2; }
    n = load(argv[1], fleet, 512);
    if (n <= 0) return 2;


    for (i = 0; i < n; i++) {
        int blockers = 0;
        Readiness r = assess_readiness(&fleet[i], &cfg, &blockers);
        counts[r]++;
        if (strcmp(READINESS_NAME[r], fleet[i].expected) != 0) {
            mismatches++;
            printf("MISMATCH %s: native=%s python=%s\n", fleet[i].id, READINESS_NAME[r], fleet[i].expected);
        }
    }
    printf("machines=%d mismatches=%d\n", n, mismatches);
    for (i = 0; i < 6; i++) printf("  %-22s %d\n", READINESS_NAME[i], counts[i]);


    /* Halt-rule self-checks (mirror tests in backend/tests/test_simulator.py). */
    printf("halt(canary,2,1)=%d expect 1\n", should_halt(0, 2, 1, &cfg));
    printf("halt(stage1,10,1)=%d expect 0 (10%% is not > 10%%)\n", should_halt(1, 10, 1, &cfg));
    printf("halt(stage1,10,2)=%d expect 1\n", should_halt(1, 10, 2, &cfg));
    return mismatches ? 1 : 0;
}
```

A.9 native/export_fleet.py

File: native/export_fleet.py | 31 lines | SHA-256 (first 16 hex): e882a6028aba04de

Exports the deterministic fleet and Python verdicts to CSV for the C harness.

```
"""Export the deterministic synthetic fleet (seed 2026, default ScoringConfig) plus the Python engine's
readiness verdicts to CSV, so native/rollout_gate.c can be cross-checked against them.


Run from backend/ (Windows):
    $env:PYTHONPATH = (Get-Location).Path
    .\\.venv\\Scripts\\python ..\\native\\export_fleet.py ..\\native\\fleet_export.csv
"""
import csv
import sys
from pathlib import Path


sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))
from osguard.fleet import PATCHES, generate_fleet  # noqa: E402
from osguard.models import ScoringConfig  # noqa: E402
from osguard.scoring import assess_fleet  # noqa: E402


out = Path(sys.argv[1])
patches = {p.id: p for p in PATCHES}
machines = generate_fleet(2026, 120)
asm = assess_fleet(machines, patches, ScoringConfig())
with out.open("w", newline="") as f:
    w = csv.writer(f)
    w.writerow(["id", "family", "version", "driver", "app", "backup", "agent", "disk", "telemetry", "latency_ratio", "error_rate",
                "vendor_hold", "compliant", "expected"])
    for m in machines:
        p = patches[m.patch_id]
        w.writerow([m.id, m.os_family, m.os_version, m.checks["driver_compat"], m.checks["app_compat"], m.checks["backup_verified"],
                    m.checks["agent_compat"], m.checks["disk_space"], int(m.telemetry_present), f"{m.health.latency_ratio:.6f}",
                    f"{m.health.error_rate:.6f}", int(m.os_version in p.hold_versions), int(m.patch_status == "compliant"),
                    asm[m.id].readiness])
print("wrote", out, len(machines), "rows")
```

A.10 native/gdb_session.gdb

File: native/gdb_session.gdb | 30 lines | SHA-256 (first 16 hex): c453e0f675526488

GDB command script used for the captured session.

```
set pagination off
set confirm off
set print pretty on
```

```
echo \n### 1. Conditional breakpoint on the readiness gate for machine vm-003\n
break assess_readiness if $_streq(m->id, "vm-003")
run
echo \n### 2. Backtrace and inspect the arguments\n
bt
print *m
print *cfg
echo \n### 3. Watch the blocker bit mask while the rules execute\n
watch blockers
continue
print/t blockers
print negative
continue
echo \n### 4. Finish the function and read the verdict\n
finish
echo \n### 5. Break on the automatic-halt rule and test the canary boundary\n
delete
break should_halt
continue
info args
print cfg->max_unhealthy_fraction
finish
echo \n### 6. Evaluate the 10 percent tolerance boundary by hand\n
print (double)1 / 10 > 0.10
print (double)2 / 10 > 0.10
delete
continue
```

Appendix B. Start-up commands (Windows)

```
cd C:\Users\aniru\OneDrive\Documents\OpenMolBench\OSGuard
.\start.ps1                 # installs on first run, builds the UI, serves http://127.0.0.1:8000
.\start.ps1 -Dev            # API on :8000 + Vite dev server on http://localhost:5173
cd backend;  .\.venv\Scripts\python -m pytest -q
cd frontend; npm test; npm run build
```